

LogForge

HLG 至 Apple Log 转换的 数学原理、源码实现与验证

从颜色传递函数到可靠媒体交付的逐层论证

研究对象	固定版本与证据
仓库	HongHuo0728 / LogForge
版本	1.2.1 (26927B)
提交	6b43a93fbd4d1457ff20b4c99a030c0e398f520a
完成日期	2026 年 9 月 29 日
正文规模	31,071 个中文汉字（含摘要，不计公式、英文和附录）
验证摘要	Release 构建通过；24 项 CTest：23 通过、1 失败
证据政策	源码事实、数学推导、历史测量和本次复测分别标识

本文为开源码的技术研究稿。未虚构作者单位、学术发表、相机认证或商业编辑器兼容性。

目录

摘要	4
Abstract	5
第一章 研究对象、证据方法与论证边界	6
第二章 信号域、颜色空间与端到端契约	8
第三章 HLG 逆函数的推导与参考曝光	10
第四章 Apple Log 编码、逆映射与可证明性质	12
第五章 YCbCr、数据范围与色度重采样	15
第六章 创意调整的模型、证明与局限	17
第七章 浮点精度、误差预算与数值验证	19
第八章 C++20 核心与外部进程架构	21
第九章 三缓冲流水线与内存上界	23
第十章 持久 CPU 工作线程与确定性	25
第十一章 CUDA 后端、资格验证与失败原子性	27
第十二章 固定帧率验证与时间轴重建	29
第十三章 音画同步、编辑列表与旋转几何	31
第十四章 MOV 原子解析与语义比较	33
第十五章 Apple Log 识别字段与元数据写入证明	35
第十六章 元数据白名单、创建时间与保真语义	37
第十七章 工具发现、执行信任与数值资格	39
第十八章 进程监督、管道死锁与取消正确性	41
第十九章 文件事务、磁盘规划与崩溃恢复	43
第二十章 GUI、CLI、队列与可复核交互	45
第二十一章 测试体系、本文复测与失败分析	47
第二十二章 性能数据、复杂度与可重复比较	50
第二十三章 从一帧样本到最终文件的实现演练	52
第二十四章 讨论、实现局限与研究结论	54
附录 A 符号、单位与复核指南	56
附录 B 全仓库文件与研究章节对应	58
参考文献与源码定位	62

图表说明与证据约定

本文的数学曲线按公开常数和当前源码公式直接计算。架构图与状态图为实现关系的抽象示意。标明“历史”的性能图使用仓库 1.2.0 数据；标明“本文复测”的结果使用 2026 年 9 月 29 日实际执行记录。所有图均不使用未经说明的虚构实验数据。

正文中的公式连续编号，图和表分别连续编号。每章末给出关键源码定位，完整逐文件范围见附录 B 和配套清单。源码超链接固定到提交，避免主分支后续变化。

阅读路径：第一至七章建立颜色与数值模型；第八至十二章解释执行和时钟；第十三至十九章解释媒体语义与可靠性；第二十至二十四章连接界面、复测、性能、实现演练和研究结论。

摘要

面向移动影像与专业后期制作之间的色彩交接，LogForge 将十位 BT.2020 HLG ProRes 素材转换为 Apple Log / BT.2020 ProRes 422 HQ。这个任务表面上只是更换视频编码，实际涉及传递函数的解释、场景曝光归一化、色度采样相位、时间轴重建、容器识别与外部工具可信执行等相互制约的问题。本文以公开仓库的固定提交 6b43a93fbd4d1457ff20b4c99a030c0e398f520a、版本 1.2.1、构建号 26927B 为研究对象，依据源码、接口、测试、构建配置、README 与版本文档，建立从信号模型到可执行程序的完整解释。

本文首先将转换分解为整数 YCbCr 解码、非线性 RGB 重建、HLG 逆光电传递、参考白归一化、可选创意调整、Apple Log 编码与有损封装，逐步推导分段函数、单调性、灰卡锚点、误差传播及不可逆边界。随后分析三缓冲浮点桥、持久 CPU 工作线程与运行时资格验证的 CUDA 实现，解释为何标准模式可按任意分量块处理，而创意模式必须配对完整 RGB 平面。最后从时间戳、MOV 原子、音频复制、工具信任、失败恢复和发布事务六个角度，论证系统正确性依赖的条件，并将仓库历史测量与本文重新执行的测试分开报告。

主要结论是：在输入解释、外部工具数值资格与输出验证成立的前提下，LogForge 实现了可追溯的颜色重编码和受约束的媒体交付流程。数学推导能够说明它为何符合所声明的函数关系，但不能证明已经丢失的传感器信息可以恢复，也不能替代不同相机与编辑软件的实际互操作实验。1.2.1 的输入准入、缺失色度位置默认值及像素旋转策略与部分历史文档不同，本文以当前代码为准明确区分。本文定位为可复核的源码研究与工程实现论文，不把项目自述、历史测试或有限抽样提升为无条件认证。

关键词：HLG；Apple Log；BT.2020；浮点视频流水线；CUDA；MOV 元数据；软件可靠性；可复现验证。

Abstract

LogForge is a native Windows application that converts BT.2020 HLG ProRes footage into Apple Log / BT.2020 ProRes 422 HQ. This monograph studies release 1.2.1 at an immutable repository revision. It derives the implemented signal transform, explains bounded CPU and CUDA processing, and examines timestamp reconstruction, metadata signaling, executable trust, cancellation, and transactional publication. Mathematical properties, repository measurements, and newly executed regression tests are distinguished throughout. The resulting argument is conditional: conformance to a declared encoding does not imply sensor-data recovery, camera-look equivalence, lossless transcoding, or universal editor compatibility.

第一章 研究对象、证据方法与论证边界

1.1 从产品需求到研究问题

视频后期中的“统一到 Log”至少包含三个不同目标。第一，给所有像素建立一致的信号解释，使同一个技术输入转换可以作用于不同素材。第二，让编辑软件正确读取色域、曲线与数据范围，不需要逐个文件修正导入设置。第三，在转换过程中保住帧数、音画同步、方向、时间码与可追溯的拍摄信息。只完成其中一项，并不构成完整的媒体交付系统。例如，写入 Apple Log 字样而不改像素，会让原来正确的 HLG 数据被错误解释；只改像素而继续保留 HLG 标签，则可能在后期触发第二次错误的传递函数转换。

LogForge 的可研究价值在于它把这些目标划分为明确模块。颜色核负责数值映射；FFmpeg 负责解码、矩阵、重采样和 ProRes 编码；MOV 分析器与识别写入器负责结构化信令；转码任务负责监督进程、验证结果和发布文件。这种划分使“如何实现”可以写成一系列具有输入、输出和失败条件的契约，而不是一条难以检查的命令行。本文沿这些契约展开，不以界面截图或主观观感代替实现依据。

研究问题可以表述为：给定一个按 BT.2020 NCL HLG 解释的输入样本及明确曝光约定，程序如何生成对应的 Apple Log 样本？当样本组成视频文件时，如何控制中间精度与存储量？当转换涉及多个线程、外部进程和磁盘文件时，如何让错误不会变成一个看似完成的最终文件？这三个问题分别对应数学层、执行层与交付层，答案彼此关联但证据类型不同。

1.2 固定版本与全仓库覆盖

本文核对了本地 Git HEAD 与远程 origin HEAD，两者均为上述固定提交。固定提交优于只记录 main 分支，因为后续修复不会悄悄改变读者正在复核的对象。正文使用“文件名+函数名”定位关键实现，附录列出全部受版本控制文件的分类、行数与哈希清单。项目代码、测试脚本、构建资源和说明文档纳入解释范围；第三方 nlohmann/json 单头文件与编译生成的 PTX 作为依赖和生成物单独登记，不把其每一行都宣称为项目自行设计的算法。

仓库同时包含不同历史阶段的说明。README 是用户入口，但实现语义的最终依据是该提交中的代码。docs/ARCHITECTURE.md、docs/METADATA.md 和早期验证记录有助于理解设计动机，却不能自动覆盖后来修改的分支。本文将当前代码、当前版本发布说明、历史测试记录依次对照；发现冲突时明确标注，而不是选取最有利的叙述。例如，旧文档要求缺失色度位置时人工声明，当前 EffectiveChromaLocation 已返回默认 left，这影响输入解释的置信度。

1.3 四类证据不能互相替代

本文采用四类证据。源码事实说明某条路径确实存在，例如曝光参数在场景线性域相乘。数学推导说明在指定条件下为何成立，例如创意亮度映射对正中性输入保持严格单调。历史实测说明仓库曾在特定机器、驱动和版本上获得某个结果。本文复测说明当前环境中实际执行的构建与测试结果。每类证据都有作用，也都有不能跨越的边界。

一个典型错误是从“CPU 与 GPU 在六组输入上输出相同”推导“所有浮点数上永远完全一致”。另一个错误是从“某个 Resolve 版本识别成功”推导“所有播放器均支持”。本文只把有限测试用于发现反例、建立工程置信度和限定兼容性范围。真正的全域结论必须来自可验证的数学前提、语言语义与实现结构；涉及外部编码器和操作系统的结论则保留相应假设。

表 1 证据层级及其允许支持的结论

证据类型	典型实例	可以支持	不能直接支持
源码核对	HLGToAppleLog 调用顺序	当前实现的计算关系	任意相机的曝光真实性
数学推导	创意曲线导数下界	指定参数域内的中性亮度单调	彩色分量均不越界
历史实测	1.2.0 性能 JSON	指定工作负载的观察结果	1.2.1 全平台性能承诺
本次复测	Release 构建与 CTest	本环境中的测试结果	新一轮商业编辑器认证

1.4 正确性是条件命题

本文把系统成功写为若干谓词的合取：输入可解释、工具可执行且数值合格、转换计算完成、时间结构满足要求、元数据一致、最终发布成功。一个成功返回值对应的工程含义，应当由这些条件共同解释。尤其要区分“输出容器合法”与“颜色正确”：容器合法的文件仍可能因矩阵、范围或传递函数错误而产生严重偏色。

$$S = I \wedge T \wedge C \wedge M \wedge V \wedge P$$

(1)

其中 I 表示输入与时间条件，T 表示工具信任和资格，C 表示颜色计算完成，M 表示媒体语义一致，V 表示验证通过，P 表示发布动作成功。这个表达式不是源码中的单一布尔变量，而是本文对 TranscodeJob 生命周期的形式化整理。它帮助读者看到：只要其中一项不成立，就不应以“转换成功”概括整个任务。

本章主要源码依据：[README.md](#)；[CHANGELOG.md](#)；[docs/RELEASE_1.2.1.md](#)。

第二章 信号域、颜色空间与端到端契约

2.1 五个不能混用的量

为避免同一个字母同时表示线性光和视频码值，本文分别使用 H 表示归一化 HLG 非线性分量， E 表示 HLG 逆函数得到的相对场景量， R 表示参考白归一化并施加曝光后的线性分量， P 表示归一化 Apple Log 分量， q 表示最终整数码值。三通道向量采用粗体记号。颜色矩阵作用于带撇号的非线性 RGB 时，不应被误写为对场景线性亮度的操作。

BT. 2020 指定的基色坐标与白点决定颜色几何；HLG 和 Apple Log 指定分量与线性量之间的非线性关系；YCbCr 矩阵决定如何把三个非线性 RGB 分量重组为亮度样分量和色差；视频范围决定这些归一化量如何映射到整数。它们是不同层次的约定。说一个文件“是 BT. 2020”并不足以判断如何显示它，因为还缺曲线、范围、采样与显示变换。

2.2 端到端运算的分解

在标准模式下，一个输入像素经历的概念映射如下：首先解码 ProRes，恢复十位 YCbCr；随后按输入范围归一化，并以明确色度相位重建三个 HLG 编码 RGB 分量；再逐分量应用 HLG 逆 OETF、参考尺度与曝光增益；最后应用 Apple Log 编码。完成所有像素后，输出端把非线性 Apple Log RGB 重新矩阵化、降采样和量化，送入 ProRes HQ 编码器。

$$\mathbf{P} = f(2^e S h^{-1}(\mathbf{H})) \tag{2}$$

这里 f 和 h 的逐分量作用是重要条件。标准模式不依赖邻域，也不需要读取同一像素的另外两个通道；因此可以把平面数据切成任意完整浮点样本块计算。可选创意模式增加依赖 RGB 亮度的映射 A ，此时同一像素的三个通道必须同时可用。两种模式由同一个最终 Apple Log 编码函数收尾，创意调整不重新定义 Apple Log 常数。

$$\mathbf{P}_{\text{creative}} = f(A(2^e S h^{-1}(\mathbf{H}))) \tag{3}$$

Signal domains and production boundaries

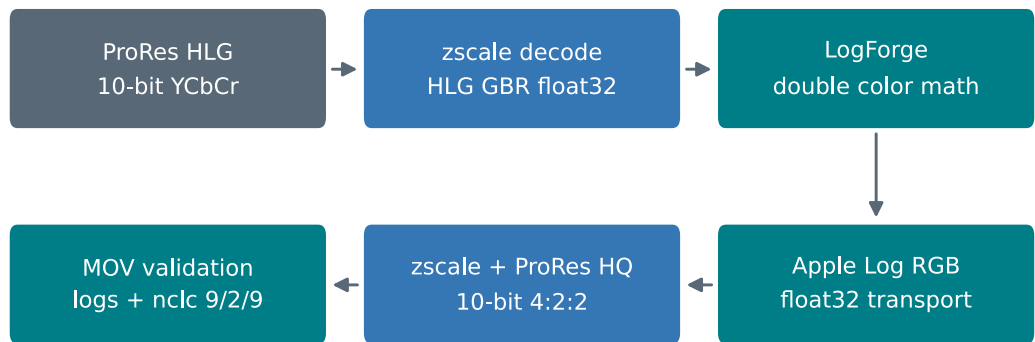


图 1 端到端数据流与信号域。蓝色为 FFmpeg 处理，绿色为 LogForge 数值结果或容器处理，灰色为源素材。沿箭头先向右、再向下、最后向左阅读；音频旁路未在此图展开。MOV 信令在编码完成后校正。

2.3 为什么不做色域变换

输入解释和输出目标均采用 BT.2020 / D65。改变曲线并不要求改变基色，因此程序没有在中插入 BT.2020 到 ACES 或其他色域的三维矩阵。Apple 提供的 ACES IDT 同时包含 Apple Log 解码与到 ACES 的基色转换，但 LogForge 使用的是前一部分的代数逆；照搬整个 IDT 会改变目标色域，偏离项目声明。

这一点也解释了为何“参照 Apple ACES 代码”并不意味着输出是 ACES。论文需要说明借用了哪一个数学对象、保留了哪一个颜色域。对应源码是 `Color.h`、`AppleLog.cpp` 和 `HLG.cpp`；它们没有引入一个隐藏的宽色域矩阵或三维 LUT。输出中 BT.2020 基色声明必须与这个事实一致。

2.4 场景解释与显示解释

HLG 系统不仅有 OETF，也有连接场景与显示的光传递关系。LogForge 选择逆 OETF 来得到相对场景量，不根据显示器峰值亮度计算尼特，也不施加显示 OOTF。这是转换目标所决定的建模选择：目标 Apple Log 仍是后期工作流中的编码，而非直接交付某个显示器的成片。

然而，相机生成 HLG 之前可能已经进行局部色调映射、降噪、锐化和曝光处理。逆 OETF 只能逆转声明的函数，不能逆转未知的相机处理链。若相机已经把高光压成同一码值，再精确的逆函数也只能得到同一个线性估计。读者应把 E 理解为“在本模型下解释的场景相对量”，不要自动理解为原始传感器电子数或真实场景辐亮度。

2.5 源码中的范围与默认解释

1.2.1 的转码器在源范围明确为 pc 时采用 full，否则采用 limited；矩阵和基色按 BT.2020 NCL HLG 显式传入滤镜。此行为减少了缺失标签导致的拒绝，但增加了默认解释的责任。输入文件若实际上采用其他矩阵，却只带有 HLG 传递标签，宽松准入并不能凭空证明它符合目标假设。

因此论文将“格式可进入程序”和“信号解释可靠”分开。当前 `UnsupportedReasons` 仅检查 ProRes 编解码器、Standard/HQ 档次与 HLG 传递标签。分辨率、时间基和时长等条件在后续处理层检查。它并没有把所有旧文档中的颜色、采样、辅助元数据限制继续作为最初准入条件。输出验证严格，并不消除输入默认值可能不符合真实录制设置这一模型风险。

本章主要源码依据：`src/Transcode.cpp`；`src/Media.cpp`；`docs/COLOR_PIPELINE.md`。

第三章 HLG 逆函数的推导与参考曝光

3.1 从分段 OETF 求逆

HLG 的低亮度区使用平方根型编码，高亮度区使用对数型编码。记常数 a 为 0.17883277， b 与 c 按下式从 a 计算。使用派生表达式而不是分别输入一组舍入值，可以减少不同实现之间不必要的常数不一致。源码在每次参考函数调用中构造 b 、 c ，编译器是否折叠常数属于优化问题，不改变函数定义。

$$a = 0.17883277, \quad b = 1 - 4a, \quad c = \frac{1}{2} - a \ln(4a) \quad (4)$$

当编码分量满足零到二分之一时，对平方根分支平方并除以三即可求得 E ；高于二分之一时，先减去 c ，再除以 a ，取自然指数后加上 b ，最后除以十二。两段对应源码 `DecodeToSceneLinear` 的条件分支。这里自然对数与二进制对数不可混用：HLG 高段使用自然底数，而 Apple Log 高段使用以二为底的对数。

$$E(H) = \frac{H^2}{3}, \quad 0 \leq H \leq \frac{1}{2} \quad (5)$$

$$E(H) = \frac{\exp((H - c)/a) + b}{12}, \quad H > \frac{1}{2} \quad (6)$$

检验连接点是最简单的实现校验。低段在 H 等于二分之一时得到十二分之一；将 c 的定义代入高段，指数项等于 $4a$ ，再利用 b 等于 $1 - 4a$ ，仍得到十二分之一。两侧导数也可以直接比较：低段为 $2H/3$ ，高段为指数项除以 $12a$ ，连接点均为三分之一。这个一阶光滑性来自常数构造，能够避免分段位置出现人为折角。

$$E' - (1/2) = \frac{1}{3} = E' + (1/2) \quad (7)$$

3.2 负值扩展的工程含义

矩阵转换和色度重建可能在边缘附近产生微小负 RGB 分量。如果立即截断到零，会改变重建滤波的结果，并让某些饱和颜色或黑边产生系统偏差。HLG.cpp 为负 H 显式使用负平方扩展。这不是把标准名义定义扩展为相机传感器真值，而是中间浮点计算的一种保留策略。

$$E(H) = -\frac{H^2}{3}, \quad H < 0 \quad (8)$$

这个扩展在零点连续，且负侧导数为负二 H 除以三，所以当 H 从更负的值增加到零时， E 也增加。相比无条件使用 H 的平方，带符号扩展不会把负输入折返成正亮度。其逆函数是负的平方根，因此源码能够对负域进行一致的往返测试。但随后的 Apple Log 有线性下限，负值保留到这里并不意味着最终编码一定保留全部负向幅度。

3.3 参考白尺度的确定

HLG 的相对场景归一化本身不等于摄影中以反射率表示的参考量。LogForge 采用 BT.2408 名义参考：百分之七十五的归一化 HLG 对应百分之百参考白。令这个位置经逆函数得到 E_w ，要求零曝光时 R_w 等于一，便得到唯一的比例因子 S 。该约定不是逐素材自动曝光估计，程序也没有从直方图推断灰卡。

$$S = \frac{1}{E(0.75)} \approx 3.774118118465752 \quad (9)$$

$$R = S E(H) 2^e \quad (10)$$

曝光参数 e 的单位是档。每增加一档，场景线性量乘二；减少一档则除二。将曝光放在线性域可以保留标准模式下非零 RGB 通道之间的比例。若在已编码 HLG 或 Apple Log 上直接加同一个常数，得到的是另一种非线性效果，不能称为同一个曝光模型。

3.4 灰卡与白卡的数值链

以百分之十八中性灰为例，零曝光时要求 R 等于 0.18，因此 E 等于 0.18 除以 S 。它落在 HLG 低段，正向编码给出 H 约为 0.3782588831；再送入 Apple Log 函数得到 P 约为 0.4882724585。这个例子提供从 HLG 到线性量再到 Log 的完整坐标，比“灰色大约在中间”更适合检查程序。

$$H_{18} = \sqrt{\frac{3 \cdot 0.18}{S}} \approx 0.3782588831 \quad (11)$$

百分之九十白卡对应 H 约为 0.7291933825，而不是 0.75。早期版本曾把 HLG 0.75 锚定到 0.9，这使整条线性尺度比现版本低十分之一。换算为曝光差是以二为底的 0.9 的对数，约负 0.152003 档。这个差异适合说明参考标定的影响，却不足以解释任意素材的大幅观感差别。

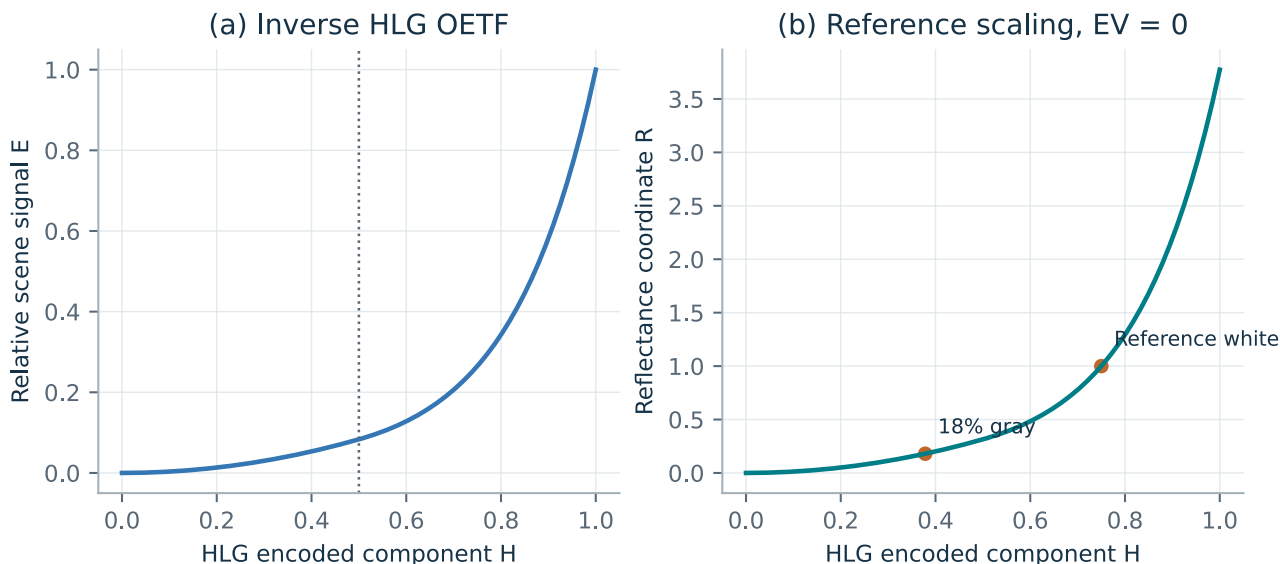


图 2 HLG 逆函数及参考锚点。左图给出名义输入域中的场景相对量，右图给出参考白尺度作用后的反射率坐标；标线为模型锚点，不是实测相机曝光。

3.5 增益、导数与噪声

局部误差传播可用一阶近似说明。若 H 发生微小扰动，在线性域的变化约为 S 乘曝光增益、再乘逆 HLG 导数。高段的导数随输入增大，因此同样的编码扰动对应不同大小的线性扰动。转换可以改变噪声的表示方式，不能使输入量化误差消失。

$$\delta R \approx S 2^e E'(H) \delta H \quad (12)$$

这也解释了为什么浮点中间量有价值：它避免在求逆和重新编码之间再施加一个狭窄整数格点。但输入本来只有十位，浮点不会创造新的测量精度；它只是让已有离散样本通过公式时不额外遭遇低精度截断。最终十位 ProRes 输出仍会重新量化，数值精度应沿整个链路讨论。

本章主要源码依据：`src/color/HLG.cpp`；`include/logforge/Color.h`。

第四章 Apple Log 编码、逆映射与可证明性质

4.1 常数、阈值与三个分支

Color.h 集中定义 Apple Log 常数, AppleLog.cpp 实现编码与解码。线性下限 R_0 为负 0.05641088, 分段阈值 R_t 为 0.01, 二次段系数 C 为 47.28711236; 高段使用 β 等于 0.00964052、 γ 等于 0.08550479、 δ 等于 0.69336945。把常数集中在头文件中, 有利于 CPU 参考函数与测试引用同一公开定义; CUDA 核中仍有独立字面常数, 因此还需要生成物哈希和数值资格验证。

$$f(R) = \gamma \log_2(R + \beta) + \delta, \quad R \geq R_t \quad (13)$$

$$f(R) = C(R - R_0)^2, \quad R_0 \leq R < R_t \quad (14)$$

$$f(R) = 0, \quad R < R_0 \quad (15)$$

分段顺序有具体意义。源码先检查高段, 再检查二次段, 最后返回零。对正常有限输入, 这与数学定义一致; 非有限输入的拒绝由外层 TransformHLGToAppleLog 负责。不能因为底层 noexcept 函数可以接受 double, 就认为它已经对 NaN 和无穷大提供完整应用级保证。

4.2 从 Apple 解码器反推编码器

高段解码器把 P 减 δ 、除 γ , 再做二的幂运算并减 β 。交换等式两侧便可解出上述对数编码。低段解码器是平方根加 R_0 , 反解得到二次式。这个推导说明 LogForge 不是凭“低对比度外观”拟合一个经验曲线, 而是以已发布解码定义为参照构造对应编码。

$$f^{-1}(P) = 2^{(P - \delta)/\gamma} - \beta, \quad P \geq P_t \quad (16)$$

$$f^{-1}(P) = \sqrt{P/C} + R_0, \quad 0 \leq P < P_t \quad (17)$$

$$P_t = C(R_t - R_0)^2 \quad (18)$$

在远离分段边界、且 R 不低于下限的理想实数运算中, 正反代入可以消去对数与指数、平方与平方根, 得到原量。这给出了往返正确性的代数依据。但它只在指定分支和定义域内成立; 低于 R_0 的所有值都被映射到零, 无法被同一个零反推出各自原值。

4.3 舍入常数造成的连接误差

Apple Log 的公开常数是有限小数。用它们计算二次段与对数段在 R_t 处的值, 两者有极小差异。因此不应在论文中宣称当前双精度字面常数形成绝对精确的连续函数, 更不应把跨接点的严格全域单调性作为未经条件化的定理。更准确的表述是: 各段内部单调, 连接处在既定数值容差内一致; 测试单独覆盖接点附近。

在对数段, 导数为 γ 除以自然对数二与 R 加 β 的乘积, 为正。在二次段, 导数为二 C 乘 R 减 R_0 , 在下限以上为正。低于下限的段为常量, 所以整体包含一个不可逆平台。本文曲线图按仓库常数实际求值, 连接误差在普通尺度上不可见, 但数值验证仍不能省略。

$$f'(R) = \frac{\gamma}{(R + \beta) \ln 2} \quad (R > R_t) \quad (19)$$

$$f'(R) = 2C(R - R_0) \quad (R_0 < R < R_t)$$

(20)

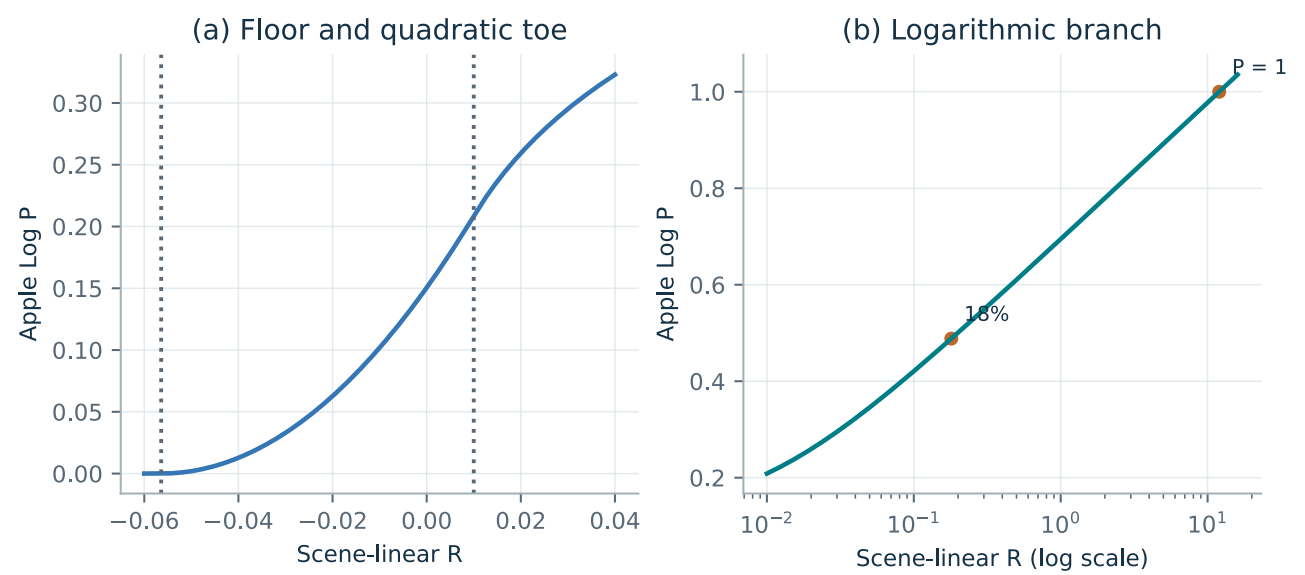


图 3 Apple Log 传递函数的阴影二次段与高端对数段。图中标出下限、连接点、18% 灰和名义高端；曲线按源码常数计算，不能解释为动态范围恢复实验。

4.4 黑点与名义白的解释

线性零经 Apple Log 编码不是零，而约为 0.1504764523。若某个后续处理把这个编码值当作普通视频黑位并重新拉到零，就已经改变了 Apple Log 数据。百分之十八灰对应约 0.4882724585，百分之九十白对应约 0.6816867959，线性量十二对应约一。它们属于归一化函数值，还没有映射到有限范围的 YCbCr 整数。

表 2 Apple Log 参考值与名义视频范围亮度码

线性量 R	归一化 P	中性亮度码 64+876P	用途
0	0.1504764523	195.8173722	线性零的编码位置
0.01	0.2085553187	246.6944592	高段入口参考
0.18	0.4882724585	491.7266737	18% 灰锚点
0.90	0.6816867959	661.1576332	90% 白卡参考
12.0	0.999999784	939.9999811	名义归一化高端

表中的码值尚未经过舍入、抖动与 ProRes 有损压缩，因此不能要求每个解码像素恰好等于表中小数。正确测试应先说明取样位置、色度边界、量化规则与误差容差。否则把有限精度结果与实数公式逐位比较，只会产生没有工程意义的失败。

4.5 不可逆性的严格反例

取两个不同线性值，都低于 R0。它们经 f 映射后相同。由函数单射的定义，f 在这个范围不可能存在恢复原输入的逆函数。类似地，最终十位量化把连续区间映射为有限整数集合，必然有不同实数进入同一个码字。这是集合与函数层面的限制，不依赖人眼是否能看出区别。

$$R_a \neq R_b, R_a < R_0, R_b < R_0 \implies f(R_a) = f(R_b) = 0 \quad (21)$$

因此，论文能够证明的是指定映射被实现以及某些结构性质成立，而不是整个转码链无损。保留更多中间精度、禁止隐式色调映射、验证编码误差都很重要，但这些措施不能逆转先前量化与相机处理造成的信息合并。对输出的评价应始终区分编码正确性、可编辑性与信息恢复能力。

本章主要源码依据：[src/color/AppleLog.cpp](#)；[include/logforge/Color.h](#)；[tests/tests.cpp](#)。

第五章 YCbCr、数据范围与色度重采样

5.1 BT. 2020 NCL 矩阵的物理位置

输入 ProRes 的三个平面通常不是 RGB，而是亮度样分量 Y 与两个色差分量。这里的 Y 由非线性 R' 、 G' 、 B' 加权得到，不能直接替代场景线性亮度。BT. 2020 NCL 的权重为 0.2627、0.6780 与 0.0593，其和恰为一。中性 RGB 三通道相等时，Y 等于该共同值，两个色差均为零，这构成检验矩阵的一个基本不变量。

$$Y = 0.2627R' + 0.6780G' + 0.0593B' \quad (22)$$

$$C_b = \frac{B' - Y}{1.8814}, \quad C_r = \frac{R' - Y}{1.4746} \quad (23)$$

逆变换先由色差恢复红、蓝，再由亮度方程求绿。FFmpegNumeric.cpp 直接用这些等式计算独立参考，而不是先调用同一个滤镜生成“正确答案”。如果生产路径和参考路径都依赖同一错误配置，二者相等也无法发现错误；这里用显式整数色块和标量算式可以降低这种循环验证风险。

$$R' = Y + 1.4746C_r, \quad B' = Y + 1.8814C_b \quad (24)$$

$$G' = \frac{Y - 0.2627R' - 0.0593B'}{0.6780} \quad (25)$$

5.2 视频范围不是简单除以 1023

十位容器提供零到 1023 的存储码字，但名义视频范围并没有占满它。亮度从 64 到 940，跨度为 876；色差中心是 512，名义总跨度为 896。输入为视频范围时，正确归一化必须扣掉偏置并使用各自尺度。把 Y 直接除以 1023 会抬高黑点并压低白点，然后这个错误会被非线性函数进一步变形。

$$Y = \frac{q_Y - 64}{876}, \quad C_b = \frac{q_{Cb} - 512}{896}, \quad C_r = \frac{q_{Cr} - 512}{896} \quad (26)$$

输出反向映射使用相同偏置和尺度。名义范围之外的码字可能承载超白或负向重建余量，但是否最终保留取决于滤镜、量化与编码器的边界行为。源码的 `aboveNominalWhite` 计数只说明归一化 Apple Log 分量超过一，不等价于已经发生码值截断。反过来，没有超过一也不能证明细小纹理在量化后仍然可分。

$$q_Y^* = 64 + 876Y, \quad q_{Cb}^* = 512 + 896C_b, \quad q_{Cr}^* = 512 + 896C_r \quad (27)$$

5.3 4:2:2 的信息结构

对宽 W、高 H 的一帧，4:2:2 包含 WH 个亮度样本，以及各 WH/2 个蓝差、红差样本，总计 2WH 个整数。以十六位容器存放十位有效值时，原始平面数据占 4WH 字节。gbrpf32le 则有三个完整平面，每个样本四字节，总计 12WH 字节。由此可见，进入浮点 RGB 后数据量是这种原始 YCbCr 存储的三倍。

4:2:2 的水平色度分辨率只有亮度的一半。把它转换成全分辨率 RGB 需要重建滤波，重新输出又需要降采样。两个过程不是一般意义上的精确互逆，尤其在彩色锐边处。Spline36 的选择属于插值核的工程决定；本文不把它描述成创造缺失色度细节。颜色正确性测试应避免不连续边界来检查矩阵，同时另外设计相位测试检查采样位置。

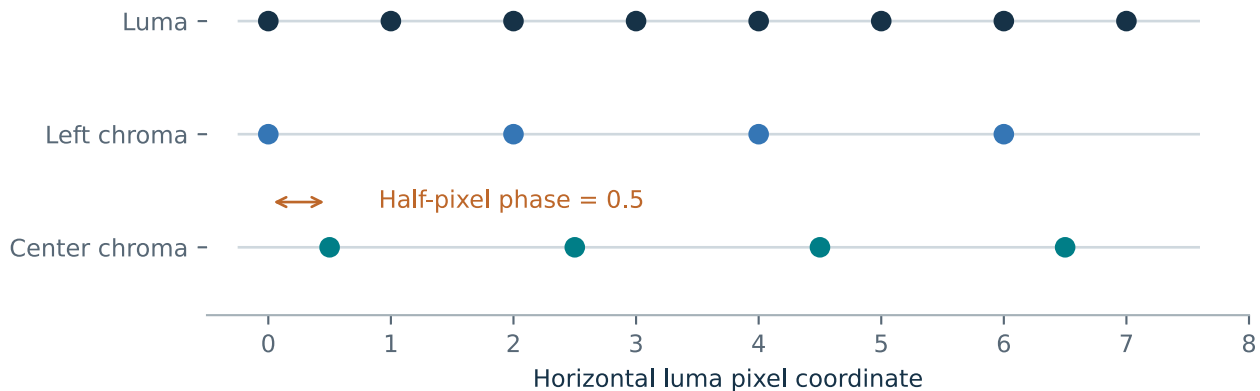


图 4 亮度采样点与 left / center 色度采样点的空间关系。半像素位移会在色度斜坡中形成可测差值；图为采样几何示意，不代表额外恢复色度分辨率。

5.4 半像素相位为什么必须测试

均匀色块无法区分 left 与 center，因为常量信号平移后仍为常量。FFmpegNumeric 使用线性色度斜坡，令每个水平亮度像素对应八个色度码的变化。若相位差半个像素，预期差值便是四个码，显著大于浮点阶段的容差。选择远离夹紧端点的位置，可以减少边界处理对这个实验的干扰。

$$\Delta q_C = k \Delta x = 8 \times \frac{1}{2} = 4 \quad (28)$$

这个实验把看似抽象的滤镜选项变成可观察的数值差异。输入端分别验证 left 与 center，输出端再实际编码并解码检查 left 相位。输出 ffprobe 可能不显示 chroma_location，因此系统保留显式声明与数值证据，而不伪造一个不存在的标准标签。标签缺失与像素相位错误是两个问题，必须分别记录。

5.5 相同 transfer 标签的作用

解码端 zscale 的输入与输出 transfer 都设为 arib-std-b67，目的是只做矩阵、范围和重采样，不重复执行 HLG 显示转换。输出端则把输入、输出 transfer 同设为 linear，使 zscale 绕过传递函数处理；实际数值此时已经是 Apple Log。随后 setparams 把输出传递声明清为 unknown，再由容器识别字段说明 Apple Log。

这里的 linear 是滤镜配置中的旁路手段，不是“Apple Log 数值属于线性光”的声明。论文若只摘抄这一选项而不解释上下文，会产生严重误导。源码注释与 FFmpeg 数值资格测试共同说明这一意图。滤镜名称和标签本身也不足以证明某个未来版本保持同样行为，所以运行时测试仍是契约的一部分。

本章主要源码依据：src/FFmpegNumeric.cpp; src/Transcode.cpp。

第六章 创意调整的模型、证明与局限

6.1 为什么创意处理单独开关

标准转换希望尽量忠实实现声明的函数关系，创意处理则主动改变场景线性亮度和色彩。将二者分离可以让用户知道输出为何变化，也让测试知道哪些参考值应该保持。ToneAdjustments 的 enabled 默认关闭；关闭时即使保存了较强的阴影提升参数，函数仍直接返回原 RGB。测试把关闭状态与标准路径逐样本比较，确保保留参数不会偷偷影响结果。

启用后的默认值为阴影提升三档、高光压缩一档、饱和度 0.85。这些数值是项目的审美选择，并非从原生 Apple Log 相机测量得到。程序把算法标识 creative-luma-v1 和有效参数写入元数据及验证报告，有助于后期追溯“这份文件是技术转换，还是还带有创意调整”。

6.2 以中灰为中心的曝光坐标

首先在场景线性 BT.2020 中计算亮度 Y 。对正 Y ，使用相对于 0.18 的以二为底对数，得到以档为单位的 x 。这样阴影和高光的控制可以在曝光距离上定义，而不是依赖输入是几位整数或使用哪种 Log 曲线。对于非正 Y ，源码选择 x 等于负六，避免对数定义域错误；这个分支属于工程约定，需要单独理解。

$$Y = \mathbf{w}^T \mathbf{R}, \quad \mathbf{w} = (0.2627, 0.6780, 0.0593)^T \quad (29)$$

$$x = \log_2(Y/0.18), \quad t = \min(|x|/6, 1), \quad w(t) = t^2(3 - 2t) \quad (30)$$

当 x 为负时，增益档数取阴影参数乘权重；当 x 非负时，取高光参数的负值乘权重。距中灰六档及以上时权重饱和到一，中灰附近权重平滑降到零。这个函数只看当前像素，不分析邻域或前后帧，因此不会因为直方图变化而自动调整参数。

$$\Delta e = s w(t) \quad (x < 0), \quad \Delta e = -h w(t) \quad (x \geq 0) \quad (31)$$

$$g = 2^{\Delta e}, \quad \mathbf{R}_{\text{new}} = g[Y\mathbf{1} + \sigma(\mathbf{R} - Y\mathbf{1})] \quad (32)$$

6.3 中灰固定点与线性饱和度

若输入是中性百分之十八灰， Y 等于 0.18， x 与权重均为零，增益为一，色差向量也为零，因此输出仍为相同灰。对任意中性灰，饱和度操作不改变它，因为三个通道各自减 Y 后都是零。对彩色输入，饱和度和缩放的是场景线性色差向量，而不是已经编码的 Apple Log 通道差。

利用三个亮度权重之和为一，可以证明饱和度部分保持亮度：先对方括号左乘权重向量，色差项的加权和为零，结果仍为 Y ；乘共同增益后亮度为 gY 。这个证明依赖没有额外截断。若某个通道经过后续曲线下限或整数边界截断，输出域中的等式就不能继续无条件套用。

$$\mathbf{w}^T(\mathbf{R} - Y\mathbf{1}) = 0, \quad Y_{\text{new}} = gY \quad (33)$$

6.4 中性亮度单调性的导数证明

令输出曝光坐标 z 为输入 x 加上档位变化。平滑阶跃函数的导数是 $6t$ 乘一减 t ，在零到一内最大为 1.5；因为 t 对曝光距离的缩放为六，档位修正对 x 的最大绝对导数为控制参数除以四。阴影和高光参数上限均为三，于是 z 的导数下界是四分之一。

$$w'(t) = 6t(1-t) \leq \frac{3}{2} \quad (34)$$

$$\frac{dz}{dx} \geq 1 - \frac{3}{6} \cdot \frac{3}{2} = \frac{1}{4} > 0 \quad (35)$$

这个结论意味着正中性亮度的先后顺序不会被创意曲线颠倒。六档之外权重为常量，导数回到一；中灰和六档边界处权重的一阶导数为零，因而拼接平滑。证明范围是正亮度坐标中的这条全局曲线，不包括任意彩色分量的排序，也不证明转换后所有细小差异仍跨越不同的量化码。

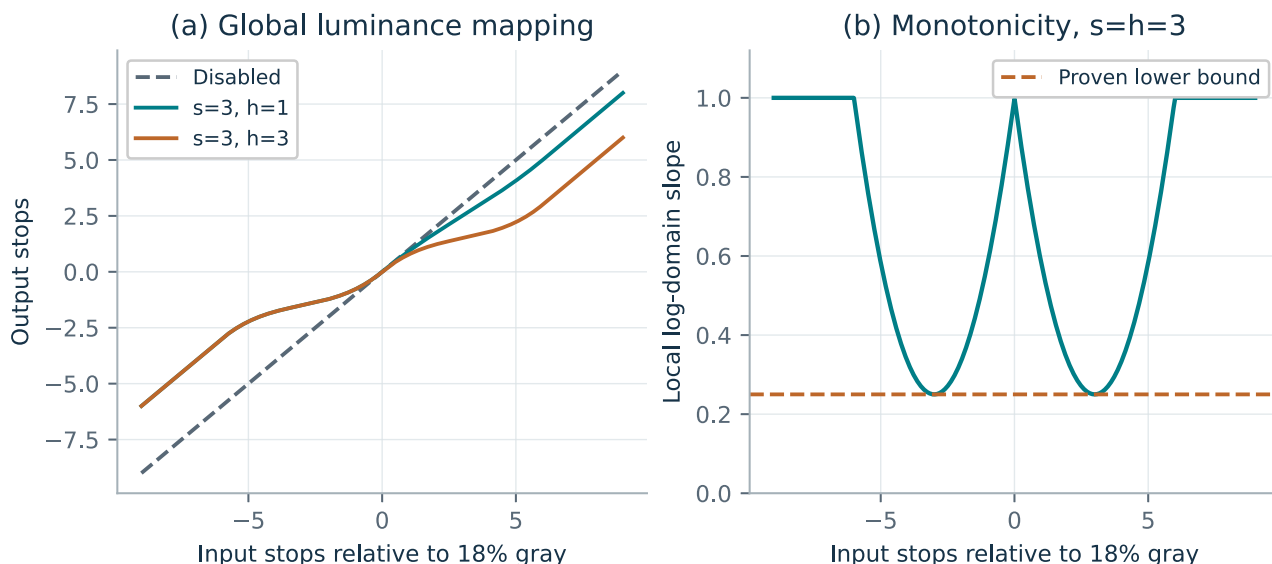


图 5 创意亮度映射与导数。上图在曝光坐标中比较关闭与启用后的曲线，下图给出参数极值时的局部斜率；水平虚线为理论下界 0.25。

6.5 饱和度越界和时间稳定性

饱和度允许到 1.5，意味着色差可以被放大。即使原 RGB 非负，沿着离开灰轴的方向放大也可能使某个分量低于零。共同增益保持饱和度为一时的通道比例，但加减亮度的饱和度操作一般不保持原通道比例。把“亮度单调”写成“所有颜色安全且色相不变”是不成立的。

时间稳定性来自参数固定与逐像素确定性：同一个输入 RGB 在同一组参数下产生同一个输出，无需保存上一帧状态。它避免了自适应曝光那种由场景统计变化引起的泵动，但不能避免源素材自身的噪声、闪烁或拍摄曝光跳变。一个确定性函数不会自动使时间序列平滑，它只是不会引入额外的内容驱动参数变化。

本章主要源码依据：[src/color/HLG.cpp](#)；[docs/CREATIVE_ADJUSTMENTS.md](#)。

第七章 浮点精度、误差预算与数值验证

7.1 精度沿管线如何分配

程序选择 float32 作为进程间传输格式，颜色表达式提升为 double，结果再写回 float32。这个组合兼顾带宽与计算稳定性：若管道也使用 double，三个平面的传输和缓存容量将再翻倍；若所有指数与对数只用低精度近似，接点、暗部和极端曝光更难与参考值一致。当前代码没有使用半精度、查表插值或快速近似内建函数替代数学表达式。

浮点误差不能用一个统一的“十位精度足够”带过。矩阵阶段、函数计算、float32 存储、再矩阵化、量化、色度重采样和 ProRes 压缩分别贡献误差，其中有的可建模为局部微扰，有的包含不可逆截断。本文使用分层预算：标量函数有自己的容差，外部矩阵参考有自己的容差，压缩后整数码有另一套容差。

7.2 复合函数的一阶敏感性

把标准变换写为 $F(H)$ ，其导数由链式法则得到。若忽略分段边界和截断，输入微小误差传播到 P 的一阶幅度由导数绝对值决定。低亮度与高亮度的敏感性不同，因此均匀输入码误差不会在最终 Log 域保持均匀。图中展示的是数学敏感性，不是对某部相机噪声的实测模型。

$$F'(H) = f'(S2^e E(H)) S2^e E'(H) \quad (36)$$

对无色中性块，输出亮度整数映射再乘 876，所以一个归一化误差 epsilon 对应大约 876 epsilon 个亮度码。以矩阵容差二乘十的负六次方为例，单纯尺度换算约为 0.001752 码；但不能据此宣称整个编码链误差都低于这个数，因为 ProRes 与重采样还有独立贡献。

$$|\delta q_Y| \approx 876 |\delta P| \quad (37)$$

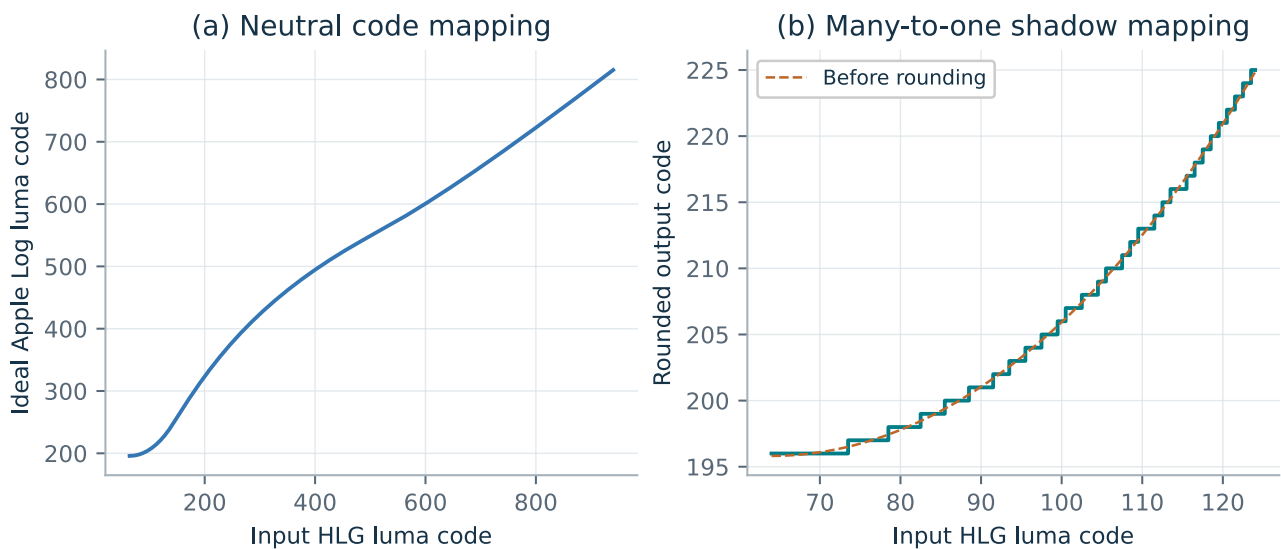


图 6 中性十位 HLG 码经过参考转换后的量化关系。暗部多个输入级可能落入相同输出整数，体现有限码字映射的不可逆性；图只模拟舍入，不包括 ProRes 压缩。

7.3 参考信号如何避免自证循环

FFmpegNumeric 构造十六个明确的整数 YCbCr 色块，包含近黑、中灰、白、超白与色差变化。其参考解码由显式矩阵给出，再通过标量颜色函数计算预期 Apple Log 值。输出侧重新生成均匀的参考浮点块，以隔离输入边缘重建影响。这样矩阵错误、范围错误和 ProRes 路径异常可以分别定位。

输入矩阵与相位误差使用二乘十的负六次方浮点容差；实际 ProRes 往返的均匀取样点使用两码阈值。tests/signal_integrity.py 的独立场景测试另有小于三码的判定，不能把不同测试的阈值合并成一个随意扩大范围。容差应绑定特定夹具、观察量与统计方式，而不是作为掩盖任何差异的总预算。

7.4 全分量计数与抽样审查的区别

SignalStatistics 在所有变换分量上累计输入最小值、最大值、输出最小值、最大值、下限截断计数与超名义白计数。对于 $W \times H \times N$ 的完整视频，正常处理应累计 $3WHN$ 个分量。这是一个有用的全量结构不变量，可以发现漏处理、重复统计或 GPU 提交错误。

$$N_{\text{samples}} = 3WHN_{\text{frames}} \quad (38)$$

但这些统计发生在最终 YCbCr 量化与 ProRes 编码之前。它们不是对最终文件每一个解码像素的审计。PixelSanity 则在第一、中间、最后一帧取九个小块，保留输入浮点数据，随后实际解码输出并比较块均值。前者覆盖全部分量但较早，后者观察最终编码但空间和时间上稀疏；两者互补，不能互相冒充。

7.5 粗粒度像素门限的正确解释

PixelSanity 的峰值块均值误差阈值为三十二码，总体平均为八码。这明显宽于均匀色块的两码资格测试，因为真实纹理、块边界与有损编码会影响小块统计。它的目标是发现通道错位、严重错误曲线或大范围数值异常，不能据此证明每个细节都准确，也不能取代严格平坦信号测试。

如果某种错误只发生在未取样的帧或区域，抽样门限可能观察不到。若错误在块内正负抵消，均值差也可能很小。因此正文把该模块称为编码后粗差检查，并保留原来的 scope 描述。更强验证可以增加帧与区域覆盖或使用全帧差异指标，但需要额外时间、存储与参考同步设计。

本章主要源码依据：src/PixelSanity.cpp; src/FFmpegNumeric.cpp; tests/signal_integrity.py。

第八章 C++20 核心与外部进程架构

8.1 模块划分与依赖方向

LogForge 使用 C++20 和 Win32，不链接 FFmpeg 多媒体库，而是启动独立 ffmpeg / ffprobe 进程，通过 rawvideo 管道、JSON 探测结果和文本进度协议交互。这个边界降低了主程序与具体多媒体 ABI 的耦合，同时把错误隔离、参数引用、进程取消和可执行文件信任变成必须自己实现的责任。

核心静态库包含颜色、媒体、工具管理、平台与转码逻辑；CLI 和 GUI 链接相同核心。GUI 负责展示和收集选项，不另写一套转换数学。这个依赖方向让 CLI 测试能够覆盖生产任务核心，也让平台相关细节集中在 Platform.cpp、StorageSafety.cpp 和 Win32 窗口模块。纯数学函数的相对独立，为未来平台迁移保留了清晰边界。

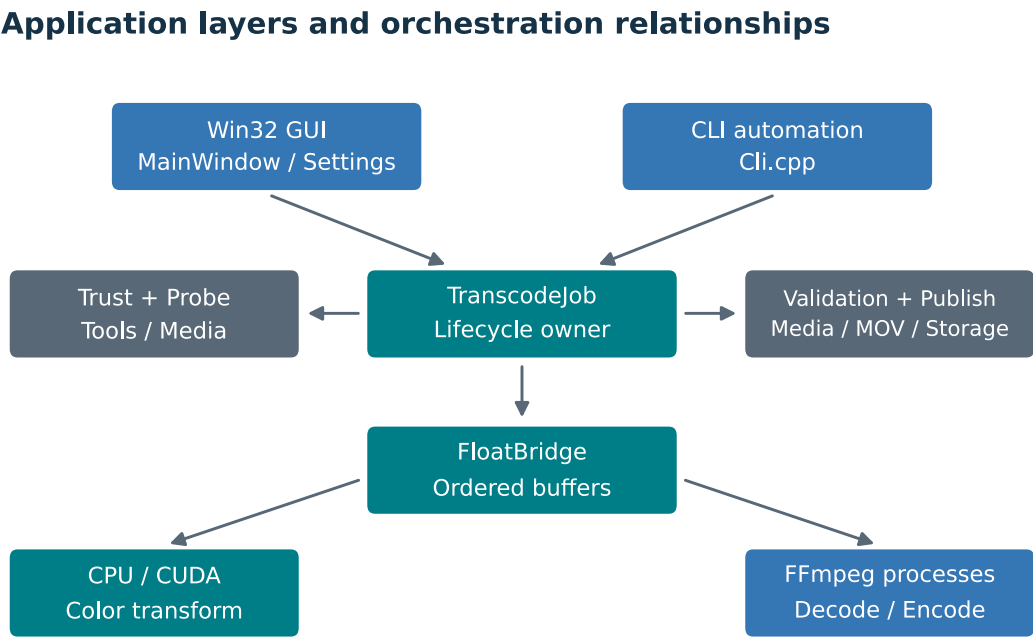


图 7 源码模块依赖关系。界面与 CLI 汇入同一任务核心；工具信任、时间验证、颜色计算与 MOV 结构处理提供相互独立的服务。箭头表示主要调用关系，不穷举每个 include。

8.2 转码任务的顺序

TranscodeJob 先重新取得工具租约并核对哈希，再检查数值资格、取消状态、参数和输入条件。随后计算方向处理方案、验证输出路径与可写日志目录、分析源 MOV、核对全部视频包节奏，并估算磁盘空间。只有这些前提成立后才创建唯一临时输出，初始化后端和进程。这种顺序让很多失败在耗费大规模转码资源之前暴露。

处理中，解码器产生浮点样本，桥接器交给 CPU 或 CUDA，编码器消费样本并复制音频。结束后依次检查进程退出码、完整样本计数、方向元数据、创建时间、Apple Log 识别和输出内容。验证报告先保存，最终文件再发布。进度回调贯穿不同阶段，但“编码达到百分之百”不等于“最终验证和发布已完成”。

8.3 两个 FFmpeg 进程的分工

解码器显式关闭自动旋转，选择第一视频流并去掉音频、字幕和数据，输出 `gbrpf32le`。必要的标准方向变换在浮点滤镜链上执行，避免先转到低精度图像格式。编码器把管道作为视频输入，同时再次打开源媒体作为音频与章节来源。它用 ProRes HQ 档次三编码视频，并使用音频流复制。

这说明“两次打开输入文件”不是重复解码所有内容：第二次输入主要供音频和容器信息使用。独立音频旁路也解释了为何改变视频像素无需重编码音频。另一方面，两输入的时钟基准必须显式对齐；否则原视频起始时间不为零时，管道重建的视频与原音频会产生错位。时间策略在后文专门推导。

8.4 参数与字符串的跨边界处理

`Platform.cpp` 将 UTF-8 与 UTF-16 分工处理：JSON、日志和多数核心字符串使用 UTF-8，Win32 路径与进程启动使用宽字符。`QuoteArgument` 按 Windows 命令行规则处理双引号与反斜杠，尤其是引号前和字符串结尾的反斜杠数量。把所有路径简单加一对引号并不足以覆盖这些情况。

这里调用 `CreateProcessW` 并传入明确程序路径，而不是依赖 shell 搜索。参数仍需按目标运行时的解析规则转义，但不需要让命令文本经过额外的命令解释器。`tests/tests.cpp` 包含空格、内嵌引号、末尾反斜杠与中文路径测试，说明这些看似外围的字符串规则属于正确执行视频任务的一部分。

8.5 异常边界和报告

颜色异常、管道异常和验证失败通过 `AppError` 或标准异常传播。读线程不能让异常直接逃出线程入口，否则进程可能终止而没有清理机会；代码将异常保存为 `exception_ptr`，再结束相关进程并由主任务重新抛出。GUI 通过结构化消息接收结果，日志保留原始外部诊断，不凭猜测翻译其含义。

验证 JSON 保存版本、输入探测、输出探测、颜色参数、信号统计、工具身份、线程分配和阶段时间。这个设计让一次转换拥有可审查的旁证文件。报告并不包含形式化证明证书，但足以让复现者知道运行时采用了哪些前提，也能区分参数错误、外部工具错误、容器错误和发布失败。

本章主要源码依据：`CMakeLists.txt`；`src/Transcode.cpp`；`src/Platform.cpp`。

第九章 三缓冲流水线与内存上界

9.1 为什么逐帧串行处理不够

若主线程先读一帧，再计算整帧，再把整帧写出，那么等待解码时颜色线程闲置，等待颜色计算时编码器缺少输入，等待编码器消费时解码器又可能受管道背压阻塞。即使每个阶段内部都开了线程，阶段之间依然存在空档。FloatBridge 的改进是在标准模式下准备三个槽位，让读、算、写可以对不同数据块同时工作。

设每块的读取、计算和写出服务时间分别为 d 、 c 、 e 。理想串行处理 K 块耗时 K 倍三者之和；在无资源争用的流水线中，填充完成后每个节拍受最慢阶段限制。真实系统还存在上下文切换、内存带宽和编码器内部缓冲，因此下式用于解释机制，不是精确性能预测。

$$T_{\text{serial}} \approx K(d + c + e) \quad (39)$$

$$T_{\text{pipe}} \approx d + c + e + (K - 1)\max(d, c, e) \quad (40)$$

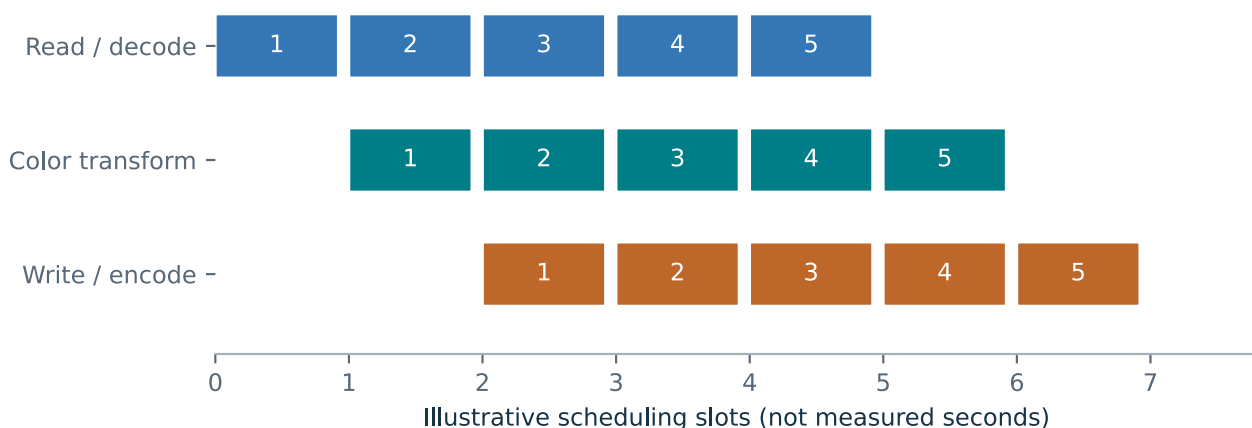


图 8 三阶段流水线的概念时序。颜色阶段始终按块顺序提交，读取与写入可与相邻块重叠。图中时间段是教学示意，不是从性能日志提取的实际甘特图。

9.2 槽位状态机

每个 Slot 有 data、filled 和 state。状态零表示可写入，状态一表示解码数据准备好，状态二表示颜色变换完成、可由写线程消费。三个执行者均以相同的环形槽位顺序前进，从而在并发情况下仍保持原始样本顺序。条件变量负责等待可用状态，互斥锁负责状态发布和观察的同步。

$$0 \xrightarrow{\text{read}} 1 \xrightarrow{\text{transform}} 2 \xrightarrow{\text{write}} 0 \quad (41)$$

这个状态循环同时表达所有权转移。读线程只有在状态零时修改数据；颜色线程只在状态一时修改数据；写线程只在状态二时读取数据。只要没有第二个越权访问者，同一槽位不会同时被两个阶段写入。源码没有为每个浮点样本加锁，而是在块级完成独占和可见性控制，因此同步成本与块数有关，而非与像素数线性同阶增加。

9.3 流结束与完整性

解码管道的一次 Read 不保证返回请求的全部字节，所以读线程循环填充缓冲。标准模式允许最后一块短于容量，但字节数必须是 float 大小的整数倍。创意模式除了完全空的结束块以外，要求每次得到完整一帧，以确保三个平面都齐全。零长度块作为结束信号经过状态一、状态二传递，写线程收到后关闭编码器标准输入。

总样本数必须能整除一帧的 $3WH$ ，才可以计算完整帧数；最终还要与预先核对的包数比较。这个双重检查分别发现“字节流被截断”和“完整帧数量与媒体声明不一致”。仅仅等待 ffmpeg 正常退出不足以替代样本计数，因为错误配置也可能生成一个语法正常但帧数不对的文件。

$$N_{\text{frames}} = \frac{N_{\text{samples}}}{3WH}, \quad N_{\text{samples}} \bmod (3WH) = 0 \quad (42)$$

9.4 标准模式的缓存规模

标准块上限为 1024×1024 个 float，即 4 MiB；三个槽位合计最多 12 MiB。当完整帧较小时，块容量取帧样本数与该上限的较小值。因此颜色桥缓存不随片长增加，也不必随大分辨率整帧增加。这个上界只针对 FloatBridge 的槽位数据，不等于整个进程树的总内存。

$$B_{\text{standard}} = 3 \times 4 \times \min(3WH, 2^{20}) \leq 12 \text{ MiB} \quad (43)$$

解码器、编码器、管道、线程栈、采样记录和 CUDA 驱动还会占用额外内存。Cadence 当前把全部包时间存入向量，也随帧数增加；所以将整个程序称为严格常量内存并不准确。更精确的结论是颜色浮点桥有固定块上界，而时间分析与外部编解码器具有自己的空间需求。

9.5 创意模式为何保留整帧

gbrpf32le 的排列顺序是完整 G 平面、完整 B 平面、完整 R 平面。设一个像素在线性索引 i ，对应三个样本的位置分别为 i 、 $WH+i$ 、 $2WH+i$ 。创意算法要计算该像素亮度，就不能只看到当前连续字节块中的一组相邻样本；相邻浮点数通常是同一平面里相邻像素，而非同一像素的三个通道。

$$\text{GBR}(i) = [p_i, p_{WH+i}, p_{2WH+i}] \quad (44)$$

当前实现为创意模式保留一个完整平面帧，桥只有一个槽位。这牺牲了三槽的阶段重叠，却避免把不同平面或不同帧错误配对。4K 帧需要 $3840 \times 2160 \times 3 \times 4$ 字节，约 94.92 MiB。可以设计更复杂的平面重组缓存来优化，但那需要改变协议或引入分通道存储，不能仅把标准模式块大小调小就得到正确结果。

9.6 失败路径也是状态机的一部分

任一线程失败时，fail 保存第一个异常、设置 abort、唤醒所有等待者，并终止解码器与编码器。等待条件均包含 abort，所以线程不会一直等一个永远不会到来的状态。JoinGuard 在异常析构时先终止外部进程，再等待读写线程退出，避免线程卡在不可完成的管道操作上。

这里不能只证明正常情况下状态零、一、二循环正确，还要证明退出条件能打破每一种等待。可靠并发代码的难点往往在后者。库函数的异常、用户取消、管道断开和部分线程创建失败，都必须收敛到清理路径，而不是留下一个等待拥有者已经消失的条件变量。

本章主要源码依据：src/FloatBridge.cpp; include/logforge/FloatBridge.h。

第十章 持久 CPU 工作线程与确定性

10.1 避免每块创建线程

FloatTransformer 在构造时创建一组持久 jthread。每次 Apply 发布像素跨度、递增 epoch，并唤醒线程。工作线程记录见过的 epoch，只有出现新任务才开始处理。主线程等待 pending 归零再汇总统计。这样大量小块不会反复支付线程创建和销毁成本，也避免把线程生命周期与单块数据生命周期混在一起。

任务采用原子 next 索引分配，Tile 为 16384。每个工作线程通过 fetch_add 领取下一片互不重叠的数据。标准模式的总数是分量数，创意模式的总数是像素数。由于每个位置只由一个领取者处理，样本运算没有跨线程写冲突；任务划分顺序变化不会改变一个样本内部的公式顺序。

10.2 创意 RGB 小块的重排

创意模式从完整帧三个平面的相同位置复制到线程私有 tile，tile 内仍按三个紧凑平面排列，调用相同标量转换函数，再写回原帧对应位置。这样每个线程只需要约三倍 Tile 个 float 的工作空间，不必复制整帧。数组的最后一个短 tile 使用实际 length 计算各平面起点，避免把固定 Tile 当作最后一块的有效跨度。

$$B_{\text{tile}} = 3 \times 16384 \times 4 = 196608 \text{ bytes} \quad (45)$$

每个工作线程约 192 KiB 的创意临时块需要计入内存解释，但它比整帧小。构造器在启动线程之前分配这些块，让分配失败在调用者异常边界内发生。若把 resize 放在线程入口而不捕获，内存不足可能导致进程终止，无法按任务失败处理。

10.3 什么情况下可以逐位一致

CPU 并行模式最终仍调用同一标量函数，每个分量的计算顺序不变，也不存在跨像素求和决定输出。因此标准输出浮点值可以与标量路径逐位比较。创意模式也只改变像素分组方式，不改变同一像素的亮度和增益表达式。tests/hardening.cpp 中的并行检查正是针对这一结构。

统计汇总需要另作说明。样本数和风险计数是整数相加，在不溢出的范围内与顺序无关；最小值和最大值针对已经通过有限性检查的样本，同样不依赖合并次序。这里没有浮点均值的非结合性问题，因为生产信号统计并不把所有像素用浮点累加求和。若未来增加均值或方差，则应重新评估归约顺序和可重复性。

10.4 小任务与线程预算

当样本规模小于两倍 Tile，Apply 直接走标量路径，避免同步开销超过计算收益。默认工作线程数按硬件并发的一半计算，并有上限；转码时更具体的线程数来自 PlanThreads。该函数同时分配解码、颜色与编码工作线程，试图避免每个子系统都把所有逻辑 CPU 占满。

对于二十逻辑 CPU 的历史环境，CPU 后端分配解码二、颜色九、编码七，另有滤镜线程。CUDA 后端把更多 CPU 预算让给解码与编码，并保留少量颜色线程用于 Auto 失败回退。线程预算不是操作系统线程总数，FFmpeg、驱动和运行时还可能创建辅助线程。论文中的线程表必须说明统计口径，否则容易把配置数量与实测进程树数量误当作矛盾。

10.5 CPU 优化的适用边界

持久工作线程能减少调度开销，但不能保证线程越多越快。小分辨率容易受固定成本限制，大分辨率可能受指数运算、内存带宽或编码器限制。多个阶段同时运行时，颜色核加速还可能使下游成为瓶颈。正确优化需要查看完整任务时间，而不是只看一个函数的运行时间。

这也是项目保留标量参考的价值。优化实现可以改变分块与调度，但用一个简洁的计算路径作为对照，有利于把性能修改与颜色定义分离。若为了速度改变数学近似、饱和行为或曝光位置，就已经不是纯实现优化，需要重新声明数值契约。

本章主要源码依据：`src/color/FloatTransformer.cpp`；`tests/hardening.cpp`。

第十一章 CUDA 后端、资格验证与失败原子性

11.1 动态驱动接口与部署边界

CudaTransformer 只从系统目录载入 `nvcuda.dll`，通过 CUDA Driver API 创建上下文、载入 PTX、分配内存、启动核函数和读取事件。程序不要求随发行包分发 `cuda`，也不在普通 MSVC 构建时编译 CUDA 源码。它选择计算能力至少 7.5 且能创建上下文的设备，嵌入 PTX 的目标为 `compute_75`。

这种结构把开发期编译器与运行期驱动分开。没有合适驱动时，Auto 可以选择 CPU；强制 CUDA 则明确报错。不能由“应用未链接 CUDA 运行时 DLL”推导“GPU 完全没有外部依赖”，因为 JIT、设备支持和驱动加载仍是必要条件。后端报告记录设备名称、驱动版本和资格结果，以便定位环境差异。

11.2 核函数的并行组织

ColorKernel 使用固定二百五十六个线程块，每块一百二十八线程。每个线程按网格步长遍历一个或多个分量，创意模式则遍历像素。局部统计存入共享内存，再通过二分归约合并为每块统计；最后由主机汇总二百五十六份记录。像素处理在设备端仍使用 `double` 表达式，输入输出是 `float`。

$$i = bT + t + kBT, \quad B = 256, \quad T = 128, \quad k \geq 0 \quad (46)$$

标准模式中每个线程的输出只依赖当前分量，因此网格步长天然覆盖且不重叠。创意模式中 `count` 是总样本数除以三，三个平面的对应下标由同一个 `i` 构造。共享归约合并计数和极值，没有跨块同步需求；块之间的总合并和同步后由 CPU 完成。

11.3 精确编译选项的意义

生成脚本使用禁止融合乘加、禁止冲零、精确除法和精确平方根选项，不启用 `fast math`。禁止 FMA 的目的之一是减少 CPU 与 GPU 表达式舍入路径差异；禁止冲零保留极小浮点数的预期行为。它们降低近似导致的偏差，但并不构成不同设备数学库逐位一致的理论保证。

`tools/compile_cuda.py` 将 PTX 写成头文件数组，并在 `manifest` 中记录 CUDA 源码哈希、PTX 哈希、编译器哈希、版本与选项。`tests/cuda_artifact.py` 重新计算两份内容的哈希，检查目标架构与关键编译参数。这避免只修改 `.cu` 却忘记更新应用真正执行的嵌入 PTX。对于生成式构建产物，源文件存在并不等于发布程序已经包含该版本源文件的行为。

11.4 六组资格样本

初始化后端时，程序生成三平面、每平面 8192 个样本的夹具，覆盖中性斜坡、带符号暗部、超白、饱和基色与固定种子的随机值。分别在标准 / 创意两种模式及负八、零、正八档曝光上运行，共六种组合。每组与独立标量 CPU 输出比较，并同时检查样本数与两类风险计数。

$$\max_i |P_i^{\text{GPU}} - P_i^{\text{CPU}}| \leq 2 \times 10^{-6} \quad (47)$$

固定种子使输入序列可重复，有助于跨驱动调查。极端曝光比只测零曝光更容易触及非线性边界和范围风险。六组全部通过之后后端才投入实际工作；这与“检测到 NVIDIA 显卡就直接使用”有实质区别。但样本覆盖仍有限，不能从有限最大误差为零得出任意输入上严格逐位一致。

11.5 主机数据的提交时刻

Apply 先把原始块复制到独立锁页主机缓冲，再异步上传、运行核函数、下载结果与统计。同步完成后检查有限性、样本数和事件结果，最后才 `memcpy` 覆盖调用方样本并合并累计统计。原始样本和累计统计在所有 GPU 操作成功前保持不变，这是 Auto 能安全回退的关键。

CUDA transaction for one host chunk

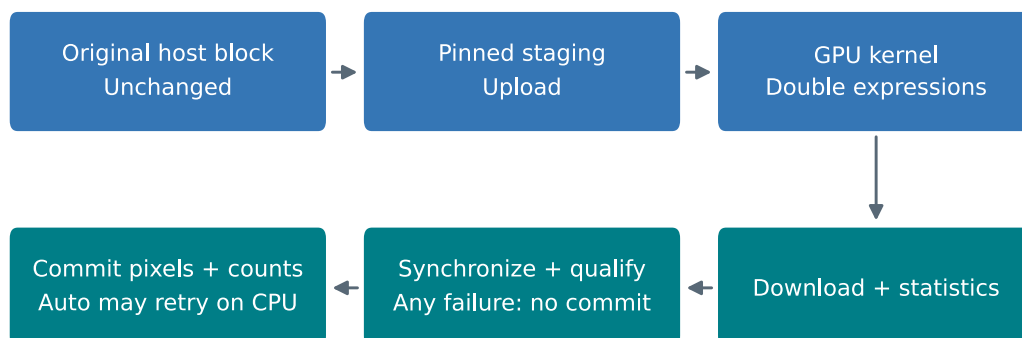


图 9 CUDA 块级执行与提交边界。只有同步、统计和错误检查全部通过，才覆盖调用方数据；失败时 Auto 可把原始块交给 CPU，强制 CUDA 则结束任务。

可以把这个性质表述为块级失败原子性：在提交点之前发生异常，主机输入仍等于进入 Apply 时的输入；因此 CPU 重新计算的是未变换数据，不会对已经编码的 Apple Log 再做一次 HLG 逆变换。若直接把设备下载写入原输入再检查错误，半完成的数据就可能被错误重试，结果具有局部双重转换风险。

11.6 CUDA 加速的真实成本

GPU 时间由上传、核函数和下载组成，还要加上锁页内存复制、上下文创建、资格验证和 CPU 编解码。短片的固定成本占比较高，纯核函数很快也可能只带来有限端到端改善。创意模式需要整帧锁页缓冲和设备缓冲，因此其内存需求明显高于 CPU 标准模式。

源码使用一个非阻塞流，按块同步提交，并不是无限深度的多流全 GPU 视频流水线。ProRes 解码与编码仍在 CPU，`zscales` 矩阵和范围处理也在 CPU。把本项目描述成“GPU 全流程硬件转码”是不准确的。它加速的是颜色数学这一段，并通过线程预算和桥接减少其余阶段的等待。

本章主要源码依据：`src/color/CudaTransformer.cpp`；`src/color/ColorKernel.cu`；`tools/compile_cuda.py`；`tests/cuda_artifact.py`。

第十二章 固定帧率验证与时间轴重建

12.1 rawvideo 丢失了哪些信息

rawvideo 管道只输送样本字节，既没有每帧原始时间戳，也没有容器的编辑列表和时基描述。编码器需要根据声明帧率重新生成视频时间。若直接把任意可变帧率输入塞入这条管道，帧间时间将被等间隔重建，可能改变运动速度与音画同步。因此固定帧率验证是此架构成立的前提，而非一个可有可无的提示。

MediaInfo 保留平均帧率和名义帧率，但 Cadence 不以两者是否相等作为最终判断。名义帧率可能是探测器猜测，平均值可能受容器时长与量化影响；真正需要检查的是所有视频包的 PTS 和 duration。这个选择解释了项目为何能够接受 29.99 等非标准但固定的节奏，同时拒绝平均值看似正常的漂移序列。

12.2 用整数时基描述节奏

记时间基为 τ 秒每 tick，第 i 包起始整数时间为 p_i ，时长为 d_i ，候选周期为 T tick。固定节奏要求每个时长接近 T 、相邻起点间隔接近 T 、累计相位接近 iT 。程序还严格要求相邻 PTS 间隔等于上一包声明的 duration，先排除内部时间表的间隙或重叠。

$$\varepsilon_d(i) = |d_i - T|, \quad \varepsilon_i(i) = |(p_i - p_{i-1}) - T| \quad (48)$$

$$\varepsilon_\phi(i) = |(p_i - p_0) - iT| \quad (49)$$

三类误差的上限均为 1.05 tick，单位是输入视频时基，不是毫秒或百分之一帧。将阈值写成毫秒而不带 time_base 会失去含义。重复或倒退 PTS、缺失或非正时长，以及间隔与上一时长不一致，都会提前失败并记录零起始包索引。

12.3 精确周期与候选周期

若所有包时长与相邻间隔完全相同，这个整数周期就是权威依据。设时间基为 n/d ，包周期为 m ，那么帧率为 $d/(nm)$ ，再用最大公约数约分。源码在乘法前检查表示范围，避免时基与长周期相乘溢出。这个分支不会为了接近 30 而把 29.99 强行改写成 $30000/1001$ 。

$$f = \frac{1}{m\tau} = \frac{d}{nm} \quad (50)$$

当周期因时钟量化不完全相同时，程序先取出现最多的包时长作为保守锚点；如果名义帧率对应周期落在观察到的时长范围内，则用它作为候选。随后对全部包检查误差。它不通过回归平均值消除累计漂移，因为那可能把真实节奏修正“拟合成正常”。

12.4 为什么必须检查累计相位

考虑每帧局部间隔只多很小的一点。单次误差都在阈值以内，但几百帧之后，起点相位误差可能超过一帧。只看局部间隔会漏掉这种持续漂移。累计相位直接比较从首帧出发的理想时钟和实际时钟，让这类系统性偏移最终越过阈值并被拒绝。

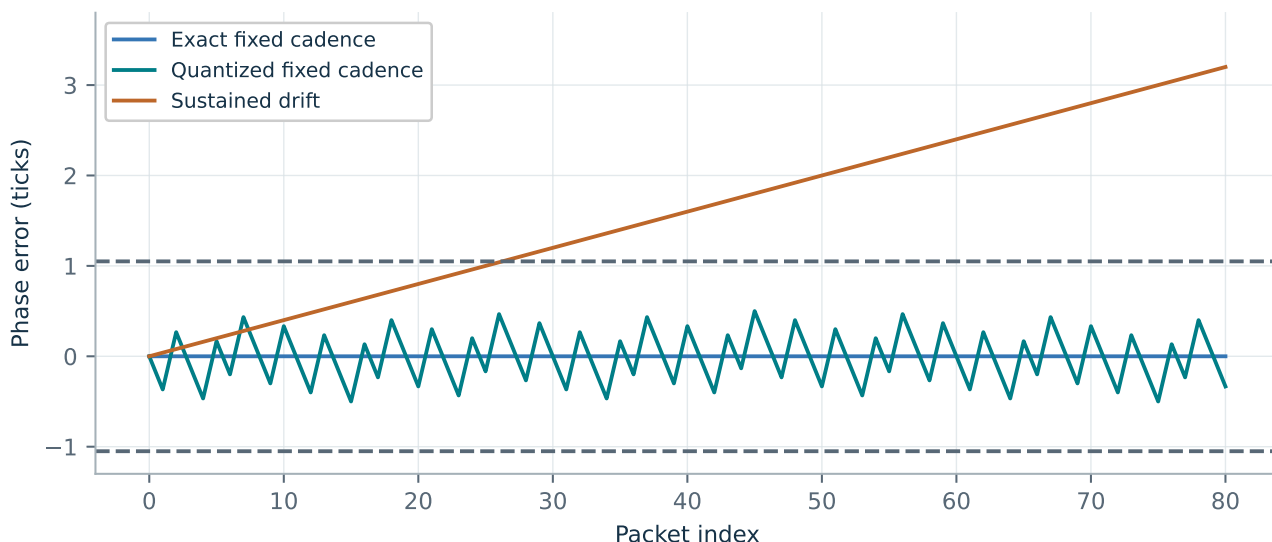


图 10 固定节奏、量化交替节奏与持续漂移的相位示例。虚线表示 1.05 tick 预算；曲线为教学构造数据，展示局部误差小并不保证累计相位小。

相反，一个正确的有理数帧率在粗时基上可能需要长短交替的整数 duration。只要累计相位始终留在预算内，这种量化抖动不应简单等同于任意 VFR。程序的政策是在可解释的固定时钟附近允许有限量化，而不是保留每个原始时间戳。论文因此使用“验证后重建的固定节奏”，避免说成“逐包时间戳完全无损复制”。

12.5 输出端再检查的必要性

输入验证通过只说明管道重建方案有依据，不说明编码器实际生成了预期节奏。输出端再次遍历全部视频包，并比较帧数、平均帧率、验证帧率与视频时长。这样可以发现错误命令、编码器同步策略或封装时间基造成的变化。帧率浮点比较阈值与时长容差在 `Media.cpp` 中明确存在，应按实际实现解读而非宣称整数层面处处严格相等。

时间扫描使用向量保存所有包，因此复杂度在包数 N 上为线性存储，构造时长频次表还包含有序映射操作。对通常只有少量不同 duration 的素材成本可控，但极长素材仍会增加前置扫描时间和内存。未来若改为流式多遍验证，可以降低内存，却必须保留候选选择与相位检查的同等语义。

本章主要源码依据：[src/Cadence.cpp](#)；[src/Media.cpp](#)。

第十三章 音画同步、编辑列表与旋转几何

13.1 音频复制不等于同步自动正确

音频使用 `stream copy` 可以避免再次有损编码，也能保持原音频有效载荷，但不保证封装后的相对时间位置自动正确。源视频可能在非零时刻开始，音频也可能带有前置延迟或编码器预热对应的编辑。管道视频重建从新的时间原点开始，因此音频时间必须以同一个原点解释。当前编码命令使用 `copyts`，并给源输入施加负的视频 `startTime` 偏移。

若源视频起点为 v_0 ，音频起点为 a_0 ，目标应保留的相对差是 a_0 减 v_0 。输入整体时间戳平移不会改变这项差值。`Media.cpp` 的输出验证直接比较输入、输出的音频相对视频起点，容差为 0.05 秒；另对存在时长的音轨检查时长差，并严格比较编码、采样率、通道数和布局。

$$\Delta_{av} = a_0 - v_0, \quad |\Delta_{av}^{\text{out}} - \Delta_{av}^{\text{in}}| < 0.05 \text{ s} \quad (51)$$

13.2 为什么保留 `movie timescale`

MOV 的电影时间基、媒体时间基和不同音轨时间基可以不同。以整数 `tick` 表示的短音频偏移，在换用较粗电影时基时可能被量化。`InspectMovTimeline` 从 `mvhd` 读取合法 `timescale`，转码器把它交给输出封装器。这个步骤是为保留时间表达能力，而不是证明所有字段逐字节复制。

仓库的 `Blackmagic` 回归使用 $500 / 48000$ 秒的起始差，即约 10.4167 毫秒。这个数字明显小于 0.05 秒，因此单看通用输出门限不足以证明它精确保留；专门回归还使用整数时间戳和音频载荷哈希检查。通用生产验证和针对性回归承担不同职责，论文将二者分开描述。

$$\frac{500}{48000} = \frac{1}{96} \text{ s} \approx 10.4167 \text{ ms} \quad (52)$$

13.3 1.2.1 对编辑列表的处理

编辑列表把电影时间映射到媒体时间，可以表达前置空段、截取和时间位置。早期版本试图按照特定模式拒绝不熟悉的列表；1.2.1 当前代码改为记录 `elst` 并由 `FFmpeg` 解释实际播放时间。视频和音频项标记为由 `FFmpeg` 解释，其他辅助流省略，已有时间码则按安全策略重新生成。

这项改变解决了 iPhone `mebx` 元数据轨具有非零媒体原点时阻断正常视频转换的问题，但不能被描述成系统已经实现所有编辑列表语义。真正执行解码和复制的仍是所选 `FFmpeg`。`LogForge` 保留包节奏、音画相对起点和输出时长检查，依靠这些结果约束任务，而不再以旧的相机专用形状白名单作为入口。

13.4 显示矩阵与角度标签

`QuickTime` 显示矩阵不只包含一个角度，还可能包含平移、缩放和透视分量。`SupportedDisplayMatrix` 解析九个整数，对允许的四种单位旋转检查精确布局；若有平移，则验证它恰好把旋转后的栅格边界移回原点。矩阵中的常规线性部分使用 16.16，最后的齐次尺度采用 2.30，不能把所有整数都除以同一个数。

按源码采用的行向量约定，坐标关系如下。对四个角点计算最小坐标，再取负值作为原点校正平移。仅凭 `ffprobe` 提供的旋转角度无法判断一个包含其他变换的矩阵与纯旋转等价，因此当前像素烘焙只对通过 `SupportedDisplayMatrix` 检查且为九十度倍数的方向路径执行。

$$x' = ax + cy + t_x, \quad y' = bx + dy + t_y \quad (53)$$

$$t_x = -\min(0, aW) - \min(0, cH), \quad t_y = -\min(0, bW) - \min(0, dH) \quad (54)$$

13.5 浮点域中的方向烘焙

对九十度和二百七十度旋转，程序交换输出宽高，并在解码滤镜的 `gbrpf32le` 阶段加入 `transpose`；一百八十度使用水平和垂直翻转。变换是样本位置置换，不需要插值缩放，因此不会额外产生重采样颜色。源 3840×2160 带竖拍方向时可输出 2160×3840 的实际竖直像素，并把最终显示矩阵设为单位矩阵。

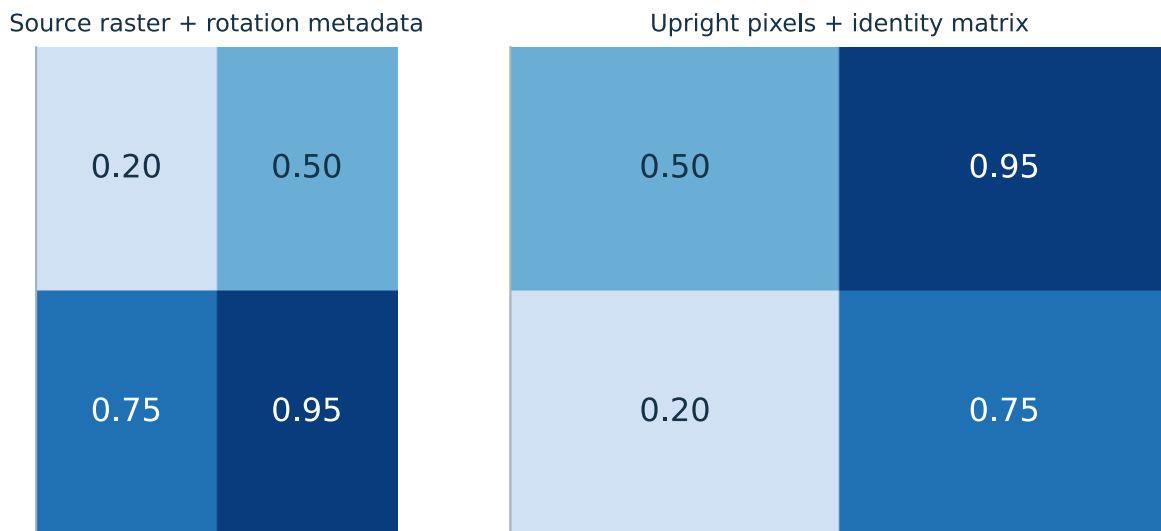


图 11 四象限夹具的像素旋转示意。90° 路径交换宽高并置换像素，单位输出矩阵使播放器不必再解释方向标签。箭头表示几何操作，不表示新的颜色变换。

在纯置换条件下，逐像素无邻域的颜色变换与旋转可以交换顺序：先旋转再计算，或先计算再旋转，对应像素的数值相同。这个性质解释了把旋转放在浮点颜色变换之前的可行性。实际程序还有 YCbCr 色度重采样，所以该交换关系仅针对已重建 RGB 与逐像素颜色核，不应扩展到任意有色度滤波的整条管线。

$$F(\Pi \mathbf{x}) = \Pi F(\mathbf{x}) \quad (55)$$

13.6 方向策略的剩余边界

对不符合标准矩阵条件的素材，当前版本不随意把角度烘焙为像素；它仍可能沿元数据保留和重封装路径处理。论文不能把旧版“拒绝不支持矩阵”的说明机械移植到当前准入代码，也不能宣称复杂矩阵已完整像素化。最终验证主要检查旋转角与头部恢复，复杂显示语义还需要专门素材和编辑器测试。

音轨布局也有类似原则：两个未标记的通道未必就是标准左右声道。程序使用 `guess_layout_max 0` 禁止推断，输出验证要求布局字符串与输入一致。保留“不知道”有时比写入一个看似合理的默认值更诚实。与色度位置默认 `left` 的策略相比，这也显示不同字段的默认政策并不完全统一，应在接口层清楚呈现。

本章主要源码依据：[src/MediaSafety.cpp](#)；[src/Transcode.cpp](#)；[tests/compatibility_v12l.py](#)。

第十四章 MOV 原子解析与语义比较

14.1 为什么需要独立容器分析器

ffprobe 能提供大量媒体字段，但不会把所有样本描述扩展和私有标识以同样方式暴露。为了研究原生 Apple Log 识别，项目实现 ReferenceMovAnalyzer，直接读取 MOV 的层级原子。分析器不解码媒体像素，它关注大小、层级、类型化元数据、颜色扩展、创建时间和显示矩阵，补足通用探测输出的盲点。

二进制容器分析首先是边界问题。普通原子有八字节头，size 为一时使用扩展六十四位大小，size 为零时延伸到当前容器结尾。解析器必须验证剩余字节足够、大小不小于头部、大小不超过父边界。把 size 直接当作可信长度读取，会让损坏文件导致越界、巨量分配或无限循环。

$$8 \leq L \leq E - O, \quad O_{\text{next}} = O + L \quad (56)$$

这里 O 是当前偏移， E 是父容器终点， L 是经验证的原子长度；扩展头还需满足十六字节。源码尽量采用“长度不大于剩余空间”的比较，避免先做可能溢出的偏移加法。结构验证与视频像素是否正确无关，但它决定元数据操作能否安全定位目标。

14.2 有界解析而非全文件载入

Reader 对单次元数据读取设置 4 MiB 上限，对原子数量设置十万上限，对嵌套深度设置二十上限，对键和编辑列表计数也做限制。mdat 不被当作需要全量载入的元数据，因此长视频的媒体载荷不会直接进入解析内存。未知小扩展可保留十六进制字节，大扩展只记录大小，避免无上限复制。

这些上限属于防御性政策，可能拒绝合法但异常庞大的元数据结构。因此“安全”与“接受所有 MOV”并不等价。对一个小型桌面转码工具，拒绝无法可靠解释的结构比无界尝试更容易维护；但论文仍需说明限制是项目选择，而不是 MOV 标准本身的通用最大值。

14.3 keys 与 ilst 的索引解析

现代 QuickTime 元数据常将键名集中存于 keys，ilst 中的整数标识引用相应索引，实际值由 data 原子携带。解析器先在同级扫描 keys，再处理 ilst，所以即使文件中 ilst 排在 keys 前面，也能正确解出键名。若只按文件顺序边看边解释，早出现的索引可能被错误当作 FourCC。

data 的前部给出类型、标志与 locale，后部是值。代码区分 UTF-8、UTF-16、带符号和无符号整数、浮点以及未知二进制。浮点值需为有限数，字符串需满足相应编码条件。不能把每段可打印字节都当作有效文本，也不能把一个 Apple API 常量名当成已经验证的文件序列化格式。

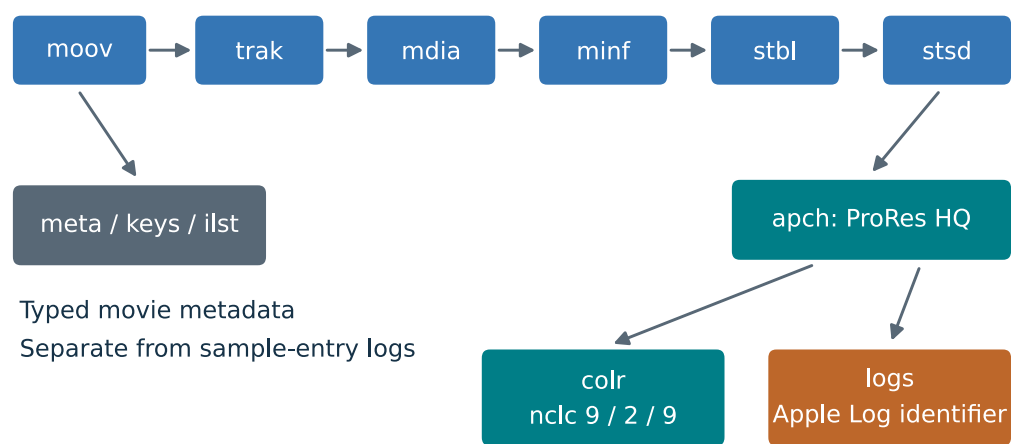


图 12 与 Apple Log 识别有关的 MOV 层级。logs 与 colr 是 ProRes 样本项的直接子原子；电影级 meta / keys / ilst 提供另一套键值元数据，不应混淆两者位置。

14.4 四字节终止符的特殊处理

真实 QuickTime 视频样本描述可在扩展列表末尾使用恰好四个零字节。它不是一个完整原子头，也不能作为任意容器尾部缺字节的借口。Reader 仅在视频扩展上下文、剩余恰为四字节且值全零时接受，并把位置记录在 video_sample_terminators 中。

这个细节展示了参考文件研究如何影响解析器：过度严格可能误拒绝合法原生结构，过度宽松又可能把截断文件当作正常。正确修复是引入具备上下文和长度条件的特例，而不是全局忽略不足八字节的尾部。识别写入器后来也利用终止符位置，把新扩展插入它之前。

14.5 语义差异与二进制差异

两个 MOV 可能因原子顺序、编码大小或媒体偏移不同而字节完全不同，但色彩解释一致。SemanticDiff 从样本项与扩展中剔除偏移、大小等不适合直接比较的字段，汇集解析后的元数据，再结合 ffprobe 媒体摘要做 JSON 差异。这个工具用于回答“哪些具有解释意义的字段不同”，而非声称两个文件二进制相同。

另一方面，语义归一化也可能忽略某些未解释字段的作用。未知私有字节被保留为原始证据，不能因为解析器没有名字就断言它们无关。项目通过受控编辑器 A/B 导入逐项隔离变量，才把 logs 标识与自动识别联系起来。分析工具提供候选差异，因果结论仍依赖实验设计。

本章主要源码依据：src/MovAnalyzer.cpp; tests/hardening.cpp。

第十五章 Apple Log 识别字段与元数据写入证明

15.1 三种声明承担不同职责

输出同时包含标准颜色描述、项目自有说明与原生参考识别扩展。colr 的 nclc 数值为 9 / 2 / 9，分别表示 BT.2020 基色、未指定传递函数和 BT.2020 矩阵；其中二不是 Apple Log 的标准枚举。logforge.transfer=Apple Log 是项目自述，方便审计但不能假定编辑器认识。logs 扩展中的 com.apple.rec2020.apple-log 则是仓库在参考文件和受控 Resolve 导入中验证的识别标识。

使用 generic log 枚举替代 Apple Log 会造成另一条曲线的声明，保留 HLG 标签则更明显与改后的像素冲突。当前方案把未知标准枚举诚实保留为 unspecified，同时通过独立验证的扩展提供更具体的信息。这种多层信令不是重复堆标签，而是在不同解释层承担不同职责。

15.2 历史识别实验的范围

docs/APPLE_LOG_IDENTIFICATION.md 记录原生参考文件来源、SHA-256、负基线、候选字段和实际 Resolve 导入结果。识别写入器的验证报告仍记录当时的软件版本及日期，并明确 this_file_imported_in_editor 为 false。它表明当前输出结构符合已验证模式，不代表每次转码都会打开编辑器重新实测。

本文保留这一区分。1.1.0 的 Resolve Studio 20.3.2.9 Windows 受控结果可以支持该配置中的兼容性证据；1.2.1 代码沿用写入器，可以说明机制未变，却不能自动得到新版编辑器或其他产品的认证。Apple Log 与 Apple Log 2 / Apple Wide Gamut 也不是同一目标，不能因为名称接近就扩展适用范围。

15.3 三十五字节扩展的结构

WriteToEncodedPartial 构造长度为三十五字节的 logs 原子：八字节普通原子头加上二十七字节 UTF-8 标识。它没有 FullBox 版本标志头，也没有结尾 NUL。这个结构来自实际参考与隔离实验，而不是从 API 名字猜测。Validate 要求唯一 HQ 样本项、唯一 logs 和唯一 colr，并检查二者均为该样本项直接子节点。

$$L_{\text{logs}} = 8 + 27 = 35 \text{ bytes} \quad (57)$$

若已有完整正确标识，写入器直接返回，实现幂等；若有冲突或重复 logs，则拒绝而非继续叠加。样本项内的 gama、mdcv、clli 等冲突扩展也会使验证失败。这个策略避免多个互不一致的颜色声明让软件按不同优先级得到不同解释。

15.4 为什么要求尾部 moov

插入新原子会增长文件。如果 moov 在媒体数据前面，移动其后字节可能改变 mdat 的偏移，需要同步更新样本偏移表，错误风险显著增加。当前写入器只接受一个位于文件末尾的 moov，并拒绝碎片化 moov。这样只需移动 moov 内插入点之后的尾部，先前媒体载荷和包偏移保持原位。

$$O_{\text{mdat}}^{\text{after}} = O_{\text{mdat}}^{\text{before}} \quad (58)$$

祖先链从 moov 到 apch 逐层检查真实包含边，不依赖固定层数。每一个祖先大小都增加三十五，普通三十二位长度若溢出则拒绝，扩展长度按原格式更新。论文能在这些结构前提下论证媒体偏移不变，而不能把该写入器描述成适用于任意布局的 MOV 通用编辑器。

15.5 尾部移动的方向与内存

要在现有文件中向后挪动一段重叠区间，必须从末尾向前复制，否则先写入的字节可能覆盖尚未读取的数据。源码使用 1 MiB 缓冲，从文件末尾逐块向插入点推进，每块写到原位置加三十五。完成移动后写入新原子，再修补祖先大小并刷盘。

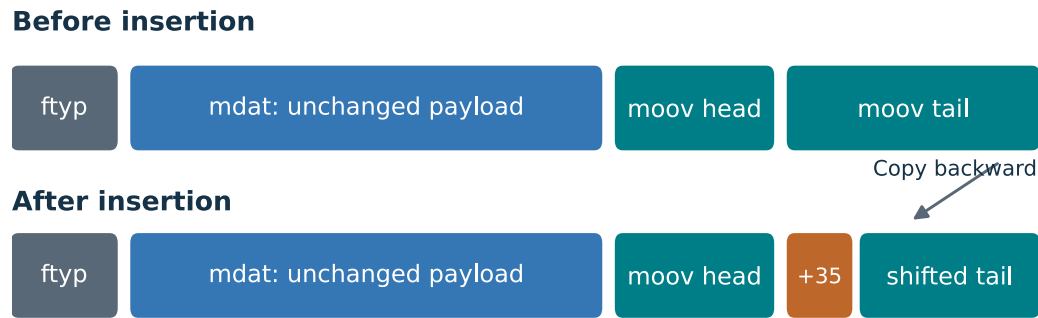


图 13 尾部 moov 中插入 logs 的示意。红色窄块是新增 35 字节；搬移从右向左读取、向后写入，以避免重叠覆盖。mdat 的位置保持不变。

在持有拒绝写入和删除的句柄期间，其他普通进程不能替换正在分析与修改的临时文件。取消可在每块移动前触发，失败后的临时文件由任务清理而不会发布。写入算法不需要为整个 moov 分配内存，也不需要重新编码视频或音频；但一旦中途失败，文件可能是不完整结构，所以事务边界必须放在最终发布之外。

15.6 保留真实来源而非伪装相机

元数据白名单可以保留真实 make、model、creation_time、timecode、language 等字段，但不会凭空生成 Apple 编码器或相机身份。视频编码器标识明确写为 LogForge / FFmpeg prores_ks。识别 Apple Log 描述的是数值解释，不是宣称该文件由某一 Apple 摄像设备原生成。

这一点对学术论证同样重要。只要通过伪造相机标签让软件显示 Apple Log，可能仍不能证明曲线、范围和编码正确。项目把颜色计算、识别字段和来源字段分开，使这些命题可以分别检验。本文也不以编辑器显示的一个名称作为全部正确性的替代证据。

本章主要源码依据：src/AppleLogIdentification.cpp; docs/APPLE_LOG_IDENTIFICATION.md; tests/identification.cpp。

第十六章 元数据白名单、创建时间与保真语义

16.1 从全量复制转向显式计划

把源文件所有元数据原样复制最简单，却容易保留已经不再成立的 HLG、HDR、Dolby、PQ 或自定义 gamma 信息。AppleLogMetadataWriter 首先禁止自动复制各范围元数据，再根据 CopyPlan 恢复有限白名单。每条保留和删除项都带有作用域、键、值与原因，从而把“元数据保真”定义为有条件保留，而不是字节全复制。

白名单包含创建时间、真实相机制造商与型号、时间码和语言。冲突检测对键名做小写化，并检查颜色相关词段；非白名单字段即使看起来无害，也可能被删除。这样的保守策略牺牲未知字段的保留率，换取更容易审计的输出解释。它并不意味着未知字段一定危险，只说明项目没有建立可靠保留契约。

16.2 作用域提升与冲突处理

MOV 普通视频流标签未必能序列化任意相机键。因此代码在安全且不与电影级同名值冲突时，把某些视频级相机字段提升到 format 范围写入 mdat。创建时间、时间码和语言保持各自专门规则。这说明复制策略不仅决定“复制什么”，还决定“写到哪里”。

输出验证按照计划里的 write_scope 查找值，而不是只在原作用域机械查找。若不存在或语义不相等，则任务失败。对默认语言 und 允许在输出缺失时等价处理，这是一个明确例外。所有例外应具有语义理由和独立测试，否则白名单会逐渐演变成不可解释的宽松匹配集合。

16.3 时间字符串的语义相等

同一时刻可以写成带 Z 的 UTC、带冒号的时区偏移或 QuickTime 常见的无冒号偏移。SameMetadataValue 对创建时间相关键尝试解析为统一微秒时间点，接受格式不同但表示相同时刻的字符串。它严格验证日期、时分秒、时区及小数部分；无法解释的无时区字符串只按原文相等处理。

$$t_{\text{UTC}} = t_{\text{calendar}} - \Delta t_{\text{zone}} \quad (59)$$

超过微秒的非零小数不会静默截断，从而避免两个本不相等的时间被误判相等。这个设计说明语义归一化应有边界：规范化可以消除表示差异，但不能通过随意舍入制造相等。论文中的时间保留也因此不是“字符串一定不变”，而是“指定字段在可解析精度内表示同一时刻”。

16.4 容器头时间与标签时间不同

电影级 creation_time 标签和 mvhd、tkhd、mdhd 中的结构化创建秒数不是同一个存储位置。某些封装路径会把全局创建时间写给所有轨道，导致原先各轨独立的头部时间丢失。PreserveMovCreationTimes 从源原子分析中按 movie 和处理器类型 / 序号匹配目标头，只修补固定宽度字段。

这个修补不增长原子、不移动 mdat、不调整样本表。若目标头只能容纳三十二位秒数而源值超出范围，函数明确失败。写后重新解析，并逐字段检查恢复值；旋转已经烘焙时同时写单位显示矩阵，否则恢复经记录的源矩阵。该流程把“保留创建信息”从单一标签检查扩展到容器结构检查。

16.5 时间码与章节

源时间码可能位于视频标签、tmcd 轨或电影标签，MediaInfo 按优先顺序收集。转码时通过明确 timecode 参数生成目标时间码轨，而非盲目复制可能覆盖新参数的旧轨。保留时间码有助于后期对齐，但它不是实际播放 PTS 的替代；一个时间码字符串正确的文件仍可能音画错位。

章节映射来自源输入，自动元数据复制被禁用后再逐项恢复标题。输出验证比较章节数量、起止时间与标题，时间容差为一毫秒。章节、时间码与音频起点是不同语义层，分别检查比只比较容器总时长更可靠。本文不把总时长相近作为所有时间信息保真的充分条件。

16.6 删除记录同样是结果

对后期工作流而言，知道某个字段被主动删除往往比悄悄丢失更有价值。报告可以说明旧 HLG 或 HDR 声明因为冲突被移除，辅助 mebx 轨因为不属于输出目标被省略。这使用户能够判断是否还需要保留原文件作为元数据档案。视频转码器不是通用媒体资产归档器，两者应采用不同的保留策略。

因此，本章所论证的是白名单内已建立语义契约的保留与冲突清除，而不是完整复制全部拍摄信息。对依赖陀螺仪、镜头校正、深度或厂商专用轨道的流程，还需要单独验证。当前仓库的范围没有授权论文把这些能力写成已实现功能。

本章主要源码依据：[src/MetadataPolicy.cpp](#)；[src/MediaSafety.cpp](#)；[src/Media.cpp](#)。

第十七章 工具发现、执行信任与数值资格

17.1 发现路径不是授权执行

桌面工具经常在 PATH 或磁盘上搜索 ffmpeg.exe，但同名文件可能来自未知来源，也可能是失效链接或被替换的程序。LogForge 把路径发现与执行授权分开：发现阶段记录候选路径、配对状态与来源；只有显式批准的路径哈希对，或来自固定哈希下载的托管工具，才可以进入能力检查。因为询问版本也需要执行程序，所以信任检查必须在 version 命令之前。

这一逻辑位于 FFmpegManager 与 ToolTrust 的边界。HasApproval 先查询记录，避免在扫描未知程序时直接执行；Acquire 则重新解析规范路径、锁定两个可执行文件、计算哈希并与批准记录比对。ffprobe 与 ffmpeg 都属于信任对象，不能只批准一个，而默认信任旁边任意同名探测器。

17.2 四元身份与租约

可执行对的身份可以抽象为两个规范路径和两个 SHA-256。只比较路径会漏掉原位置文件替换，只比较哈希而忽略配对路径又难以表达用户实际批准的安装位置。ToolLease 持有两个拒绝写入与删除的句柄，在相关进程执行期间持续存在，以缩小检查与使用之间被替换的窗口。

$$\mathcal{I} = (p_f, p_p, H(f), H(p)) \quad (60)$$

这属于特定操作系统共享模式下的执行身份约束，不是对整台机器免受管理员级攻击的证明。哈希证明字节与批准对象一致，不证明其作者可信或没有漏洞。论文只据此论证“当前执行的工具与记录绑定”，不会把批准机制写成恶意代码检测器。

17.3 快速搜索的有限预算

Quick 搜索共享三千毫秒文件系统预算。托管和已保存路径优先，若找到已批准且数值验证通过的工具，则不再遍历后续来源。否则剩余预算覆盖程序附近、PATH、注册表 App Paths、包管理器实际安装目录、有限深度常见目录与现有 Windows Search 索引。哈希与数值资格耗时独立记录，不消耗这个文件系统预算，因此三秒不是整个启动过程的绝对上界。

DiscoveryHelper 在同一个程序的隐藏入口中运行，父进程监督它。这一点用于处理可能阻塞的磁盘或索引查询：把操作放在普通后台线程不能保证立即取消一个卡住的系统调用，而独立受监督进程提供了更强的退出边界。Deep 搜索只在用户明确选择后才枚举本地可访问磁盘，并跳过重解析、离线与保护区域。

17.4 搜索结果如何保持诚实

搜索记录实际目录数、跳过数、候选来源、结束原因和诊断，不把超时或索引不可用描述成“系统没有 FFmpeg”。候选安全上限为二百五十六，常见目录遍历还有深度与条目限制。发现数量是当前范围内的观察，不是全盘完整资产清单。

路径规范化还处理大小写、长路径和挂载卷。程序使用 Windows 已知文件夹定位安装位置，不把用于测试的 LOGFORGE_DATA_DIR 误当作所有真实用户目录。测试注入根目录与虚假工具来检查搜索是否超范围，以及未知程序是否被意外运行。这些测试不需要扫描用户所有磁盘就能验证关键分支。

17.5 能力检查与数值检查

允许执行之后，程序检查 ProRes 解码器、prores_ks、gbrpf32le、yuv422p10le 以及 zscale、format、setparams 等能力，并运行小型编解码冒烟测试。即使这些名称都存在，也不能说明矩阵、范围和采样相位符合要求，因此还必须通过独立数值信号测试才能获得 numericallyVerified 状态。

Execution admission is a sequence of distinct gates

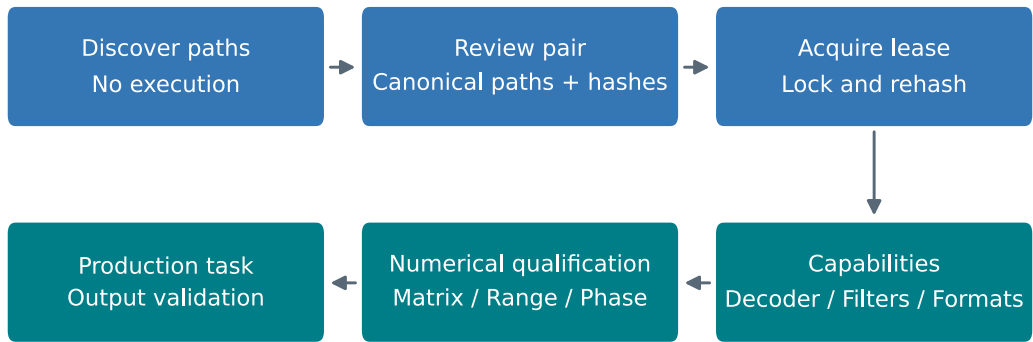


图 14 外部工具的准入链。每一关解决不同问题：身份绑定、功能存在、数值行为和实际任务验证。通过前一关不自动意味着后一关成立。

TranscodeJob 再次检查资格标志及租约哈希，防止调用者绕过探测流程直接提交一个仅兼容但未经数值验证的工具。这种纵深结构使 UI 和 CLI 的展示状态不是唯一防线。核心自身也验证执行前提，从而降低未来入口增加时遗漏检查的风险。

17.6 固定下载与供应链边界

GyanReleaseProvider 固定版本、HTTPS URL、归档根目录和 SHA-256，安装器限制下载体积、检查完整性、最多重试三次，并先在暂存目录验证再移入托管位置。归档条目需位于预期根目录，不允许绝对路径、上级路径或驱动器标识。旧损坏安装会保留为带所有权标记的备份，而不是无条件覆盖。

固定哈希避免“latest”内容变化造成不可复现，但也意味着升级必须显式更新并重复资格测试。HTTPS 提供传输保护，固定哈希提供内容绑定，运行时数值检查提供行为约束，三者不相互替代。本文没有为安装器执行新的在线下载，相关论述依据源码与已有测试，不虚构供应链审计结论。

本章主要源码依据：src/ToolTrust.cpp; src/FFmpeg.cpp; src/Discovery.cpp; src/DiscoveryHelper.cpp。

第十八章 进程监督、管道死锁与取消正确性

18.1 受监督启动顺序

Platform.cpp 创建子进程时先设为 `suspended`，把它加入具有 `kill-on-close` 属性的 Windows Job Object，再恢复主线程。先启动后加入监督存在短暂逃逸窗口，先挂起再绑定则让进程在执行工作前就属于监督域。主程序异常退出时，Job Object 句柄关闭会触发所属进程清理。

STARTUPINFOEX 明确限制继承句柄，只允许子进程标准输入、输出和错误所需的三个句柄。若让所有可继承句柄进入子进程，解码器可能持有编码器管道的某个末端，即使真正写入者退出，读取者也观察不到 EOF。这种隐蔽的句柄泄漏通常表现为任务完成后永久等待，不能靠简单超时数字掩盖。

18.2 标准错误也需要持续排空

管道容量有限。若父进程只读 `stdout`，而子进程大量写 `stderr`，`stderr` 填满后子进程会阻塞，继而停止产生 `stdout`，父进程又在等 `stdout`，形成循环等待。转码器分别开线程排空解码错误、编码错误与编码进度；RunProcess 也并发读取标准输出和错误。

用等待图可以把问题写为父进程等待子进程输出，子进程等待父进程释放错误管道容量。消除这个环需要独立消费者，而不是增加一点缓冲。当前实现还限制每行和累计保留大小，避免一个不换行的异常输出耗尽内存；有回调时可以继续流式处理，而不必把所有输出永久留在字符串里。

18.3 父进程退出不等于管道关闭

一个工具可以启动子孙进程，子孙继承管道句柄后，原始工具退出仍可能留着管道。RunProcess 的监督循环条件包含原始进程运行状态，以及 `stdout / stderr` 读取线程是否完成。因此即使原始进程已退出，取消与截止时间仍继续生效，直到管道真正结束。

tests/process_fixture.cpp 专门构造后代持有管道、长行输出和不同退出码等行为；reliability_process 验证异常传播、时间限制和后代终止。这样的夹具比仅对正常 ffmpeg 调用测一次取消更有针对性，因为它直接覆盖曾导致挂起的资源生命周期关系。

18.4 取消与清理的顺序

用户取消设置原子标志，转码取消观察者以短周期检查并终止两个 Job Object。受阻读写因此退出，异常或取消状态回到主任务；随后 join 所有线程，删除属于当前任务的临时输出。关闭窗口走相同路径，避免界面消失而后台编码器继续运行。

清理顺序必须是先让外部阻塞操作有机会结束，再等待线程。若先 join 一个正在读永不关闭管道的线程，后面的终止语句永远没有机会执行。源码多个 RAII guard 体现同一原则，并覆盖线程仅创建一部分就发生异常的情况。析构安全性在并发程序中应当作为设计前提，而非最后补上的错误处理。

18.5 退出码与异常的优先级

任务既检查用户取消，也检查进度线程、错误读取线程、颜色处理异常、解码退出码、编码退出码和帧数。一个进程返回零不能抹去读取器异常；帧数正确也不能抹去数值失败。保留第一个异常有助于报告根因，但日志仍记录退出状态和外部诊断，便于分析连锁失败。

进度数据本身也按有界文本处理。`frame`、`fps`、`out_time_us` 与 `speed` 分别解析，无法解析的单个值不应直接产生虚假的精确估时。任务阶段允许不确定进度，例如写识别字段和最终验证。界面应展示阶段名称，而不是把所有工作硬压成一个看似连续且始终可信的百分比。

18.6 可证明和不可证明的监督性质

在操作系统按文档实现 `Job Object`、句柄共享和管道语义的前提下，可以论证子进程属于同一清理域、读写线程有取消解除路径、未通过验证的临时媒体不会被主动发布。不能仅据这些机制证明任意驱动挂死、内核异常或断电时所有系统资源都即时回收。本文讨论的是应用层可控制的失败模型，并明确把操作系统服务作为假设。

这种边界不是削弱结论，而是让结论可检验。若一个论文声称“保证永不卡死”，却没有说明系统调用、驱动与硬件的假设，实际上比一组具体且受测的退出条件更难复核。LogForge 的监督设计适合用资源所有权和状态转移说明，而不适合用无限制的可靠性形容词描述。

本章主要源码依据：`src/Platform.cpp`；`src/Transcode.cpp`；`tests/process_fixture.cpp`；`tests/reliability.cpp`。

第十九章 文件事务、磁盘规划与崩溃恢复

19.1 最终路径与临时路径分离

TranscodeJob 要求目标为新 MOV 文件，并在同目录创建带进程号和时间计数的唯一 partial。转换、元数据修补和验证都作用于 partial，目标文件名只在全部条件通过后获得内容。CREATE_NEW 预留临时路径，避免无意占用已有文件；最终发布使用不带 REPLACE_EXISTING 的 MoveFileExW。

预先检查目标不存在仍不足够，因为用户或另一进程可能在长时间转换期间创建同名文件。最后的无覆盖移动再次由系统执行冲突检查，从而关闭“检查时不存在、写入时覆盖别人文件”的常见竞态。发布失败时任务仍失败，不能因为编码工作已经完成就覆盖目标或偷偷换名。

19.2 发布不变量

可以把最终文件存在与本任务主动发布之间的关系写成安全条件：本任务只有在验证通过、报告保存成功且取消未发生时才尝试发布。这个条件不意味着目录中任何同名文件都由本任务生成，因为其他进程仍可能创建它。论证对象应是任务自身的写入行为，而不是整个共享目录的全局状态。

$$\text{PublishAttempt} \Rightarrow \text{Validated} \wedge \text{ReportSaved} \wedge \neg \text{Cancelled} \quad (61)$$

Validation precedes final-file publication

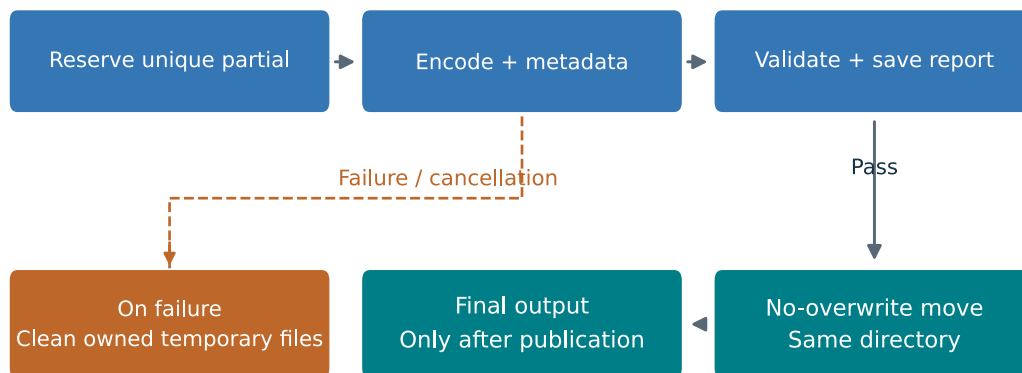


图 15 任务状态与发布边界。编码完成后仍需元数据修补、输出验证和报告持久化；任何失败回到清理路径，只有最后的无覆盖移动使临时媒体成为最终交付文件。

同目录移动有利于保持同卷语义和避免整文件复制。WRITE_THROUGH 表达写入持久化意图，但本文不把它等同于在所有文件系统、缓存设备和突然断电场景中拥有数据库级事务保证。程序使用的 API、文件系统行为与硬件耐久性不同层次的问题。

19.3 磁盘需求是启发式估算

空间预检以 1080p、约 29.97 fps 的 ProRes HQ 约 220 Mb/s 为基准，按像素数、帧率和时长比例扩大，再乘 1.75 余量，加音频估算和固定余量；需要旋转重封装时考虑第二份拷贝。由于场景复杂度和编码器行为会改变实际码率，这只是提前发现明显不足的规划值。

$$B_v \approx \frac{220 \times 10^6}{8} D \frac{WH}{1920 \cdot 1080} \frac{f}{30000/1001} 1.75 \quad (62)$$

预检通过后仍可能因其他进程占用空间、设备断开或实际码率偏高而写入失败。因此运行时写错误必须继续致命处理。tests/v120.cpp 通过 JobIOHooks 注入低可用空间和报告写失败，验证失败不会发布最终媒体。把这些钩子放在测试注入接口，而不放进普通用户环境变量，可减少生产路径被无意改变。

19.4 临时文件所有权

崩溃后留下的 partial 不能仅凭文件名匹配删除。StorageSafety 为任务保存日志，记录进程号、进程创建时间、绝对路径、卷序列号、文件索引和文件创建时间。恢复时先确认原所有者死亡或进程号已被新进程复用，再检查当前路径对应的文件身份仍与记录相同。

$$\text{MayDelete} = \text{OwnedJournal} \wedge \text{DeadOwner} \wedge \text{SameFileIdentity} \quad (63)$$

进程号本身会重复使用，因此必须加入 process_start。路径本身也可能指向后来替换的新文件，因此必须加入文件身份。无法打开进程若是访问被拒绝，不被视为死亡证据；身份不一致时也不删除。这个保守策略优先避免误删用户媒体，代价是某些不确定残留可能留待人工处理。

19.5 状态存储的并发锁

settings 与 trust 的修改是读、改、写三步，单靠最终原子替换不能防止两个进程相互丢失更新。StateLock 对数据目录下锁文件的一段字节持有独占锁，结合进程内 mutex 串行化读改写，再用同目录临时文件替换正式文件。Windows 在进程退出时释放文件锁，有利于崩溃后恢复。

保存偏好只更新归自己负责的字段，保留独立保存的 FFmpeg 路径；保存路径也不覆盖其他偏好。这个字段级合并策略减少不同功能之间互相覆盖。但同一偏好由两个窗口同时修改仍采用最后保存结果，程序不是协同实时编辑器。论文应明确并发更新的粒度，而不是简单说“设置线程安全”便结束分析。

19.6 留存策略的范围

维护函数只清理受识别的日志名与安装目录，检查年龄、所有权和重解析点；不递归搜索用户媒体目录删除所有 MOV。日志通常有三十天留存阈值，带标记的旧安装暂存目录有独立阈值。无日志的旧 partial 不因名称相似就获得删除授权。

这种设计把便利性与可证明所有权绑定。对于大体积媒体任务，清理功能本身可能比转换失败更危险；所以恢复和留存不应作为“简单删除临时文件”的附属代码看待。它们需要与发布机制一样严谨的路径、身份和并发假设。

本章主要源码依据：src/StorageSafety.cpp; src/Settings.cpp; src/Transcode.cpp; tests/v120.cpp。

第二十章 GUI、CLI、队列与可复核交互

20.1 一个后台任务拥有者

MainWindow 以一个后台 worker 承担探测、下载、验证、转码或队列操作。busy 状态禁用会与正在执行任务冲突的控件，参数在 UI 线程形成不可变任务选项后交给工作线程。这样用户在处理中改变设置，不会让同一个视频前半段用一组参数、后半段用另一组参数。

工作线程通过 WM_APP 自定义消息发送事件，事件包含类型、进度、媒体、工具、错误或报告。堆对象的所有权通过 PostMessage 转移，UI 接收时用 unique_ptr 消费。这个模式把跨线程状态变化集中到消息处理，而不是让后台线程随意操作窗口控件。关闭过程中还需处理未消费事件与线程结束，防止生命周期错位。

20.2 布局由实际文字测量驱动

主窗口以一个布局拥有者计算控件矩形和换行结果，文字测量与绘制使用一致断行，避免背景绘制与控件重复写同一状态。DPI、窗口大小、语言、主题或状态变化都会触发布局更新。滚动视口保证小窗口和高缩放下仍能到达底部控件，键盘焦点移入不可见区域时会调整滚动。

Ui.cpp 负责字体、画刷、颜色、原生控件外观和 WIC 截图；系统高对比度优先于项目主题。DWM 标题栏属性在可用时请求，不依赖私有暗色 API。界面是否可用不是由颜色漂亮决定，而是错误诊断能否读完、按钮是否可达、焦点是否可见、状态是否由唯一来源呈现。

20.3 设置草稿与取消语义

SettingsWindow 使用 draft 保存编辑中的参数，只有 Save 才调用 SettingsStore 并将 accepted 标志置真。Cancel 直接销毁对话框，不把草稿写回。创意开关改变草稿并刷新控件可用状态，语言和主题选择也先留在草稿。这个“提交前不影响运行配置”的模型与文件发布事务在概念上相似，但作用域是用户偏好。

测试会先打开并修改草稿，再取消，确认偏好没有泄漏；随后保存另一组参数，检查落盘和实际应用。使用一份可变全局设置直接绑定控件可能让取消只关闭窗口却不撤销已发生修改，因此草稿对象是可观察行为所需要的结构，而不仅是代码组织偏好。

20.4 稳定消息键与翻译

Messages.inc 通过宏表同时定义消息标识与双语文本，Localization.cpp 检查键唯一性、条目完整性和占位符集合匹配。工作线程传递 TextId 与参数，UI 按当前语言翻译；JSON 中保留稳定错误码，日志与原始工具诊断保留英文。错误码适合自动分析，译文适合用户阅读，二者承担不同目标。

占位符替换采用单遍处理，插入的路径或错误文本不会再次被当作模板。如果文件名恰含有类似花括号零的字符，它应当原样显示，而不是触发二次替换。这个细节也属于输入边界：用户可控字符串只能是数据，不能在格式化后升级为控制模板。

20.5 串行队列与逐项结果

RunQueue 为每个输入生成独立输出名，逐项探测并调用同一 TranscodeJob。一个项目失败会记录错误和失败报告，然后继续下一项；用户取消则停止后续处理。结果包括成功数、失败数、剩余数与取消状态。队列没有同时启动多个媒体任务，因此不会把原来的三阶段资源预算再乘以文件数量。

默认输出名基于源文件 stem 加 AppleLog 后缀，不做自动覆盖。不同来源同名文件可能产生目标冲突，此时对应项目失败而不是覆盖先前结果。这样的行为需要在批处理解释中明确：队列提高操作效率，但不消除命名碰撞和磁盘规划问题。

20.6 CLI 的自动化价值

Cli.cpp 提供 detect、probe、convert、batch、qualify-cuda、analyze 等入口，参数解析先验证命令形状和创意开关关系，再发现或执行工具。只指定创意参数却不启用 tone 会被拒绝，避免用户误以为调整已经生效。曝光和创意数值必须完整解析、有限且在范围内，带尾部垃圾的数字不会被部分接受。

CLI 与 GUI 共享核心，使数学、时间与发布规则保持一致；但二者可用的曝光界面范围不同，GUI 为离散负四到正四档，CLI 接受负八到正八档。复现论文实验时应写清入口和全部参数，而不是只写“默认设置”。同一个默认词在不同时间、保存状态或界面中可能指向不同配置。

本章主要源码依据：src/MainWindow.cpp; src/SettingsWindow.cpp; src/Ui.cpp; src/Localization.cpp; src/Cli.cpp; src/Queue.cpp。

第二十一章 测试体系、本文复测与失败分析

21.1 测试不是一个总分

仓库测试覆盖数学、媒体解析、工具信任、进程监督、时间节奏、MOV 识别、并行一致性、设置、资源和真实编解码。把这些测试合并成一个通过率有助于快速概览，却不说明每个研究问题获得多少证据。一个纯数学测试通过并不能证明音频同步，一个 GUI 截图通过也不能证明像素符合公式。本文按契约分组，而非只展示绿色通过标记。

tests/tests.cpp 包含颜色参考点、密集往返、参数和有限性；hardening.cpp 检查时间、信任、MOV 与并行路径；reliability.cpp 配合 process_fixture 和 fake_ffmpeg 检查异常进程与未知工具不执行。application.cpp 覆盖偏好、发现和资源；identification.cpp 直接检查识别字段写入结构；v120.cpp 进一步注入存储与后端故障，并覆盖队列与元数据。

21.2 实际媒体测试的角色

integration.py 生成可控媒体并检查真实输出，signal_integrity.py 以整数色块独立验证颜色，sustained.py 处理连续 240 帧 4K120 并检查内存和抽样亮度。compatibility_v121.py 用生成结构覆盖竖拍像素、单位矩阵和音频偏移。它们测试的是生产核心的端到端行为，而非只调用一个简化替身。

gui_smoke.py 与 layout_snapshots.py 需要交互桌面和真实窗口，用于不同语言、主题、DPI、操作路径和截图检查。resolve_identification.py 需要实际 Resolve 脚本连接和合适参考，属于编辑器互操作实验。本文没有把后两类历史结果当作本次重新运行结果，也没有以模拟窗口或伪造编辑器返回值替代它们。

21.3 本次构建与完整 CTest

2026 年 9 月 29 日，本文在当前 Windows 工作区执行 Release 构建，所有配置目标构建成功。随后运行当前 build 配置中的二十四项 CTest，包括实际 CUDA 资格、媒体转码、竖拍兼容、信号、发布和持续 4K120。结果为二十三项通过、一项失败，总测试墙钟时间约 195.71 秒。另执行 CUDA 生成物一致性脚本，通过源哈希、PTX 哈希、目标与精确编译选项检查。

测试日志与 JUnit XML 随论文资料保存。XML 顶层 time 字段按工具输出为整数秒，逐项耗时保留小数；正文总时间取 CTest 终端总结。这里没有修改源码或测试后重跑来获得全绿结果，固定提交保持不变。失败本身是研究对象的一部分，不能为了“证明工具正确”而隐藏。

表 3 本文实际运行的完整 CTest 结果

测试名称	结果	耗时 / 秒
robustness_v120	通过	0.940
cuda_qualification	通过	0.133
apple_log_identification	通过	0.557
reliability_process	通过	1.762
reliability_discovery	通过	4.128
reliability_cli	通过	1.597

测试名称	结果	耗时 / 秒
hardening_cadence	通过	0.085
hardening_security	通过	0.540
hardening_mov	通过	0.051
hardening_parallel	通过	2.486
color_math	通过	0.192
media_metadata_validation	通过	0.015
ffmpeg_platform	通过	1.007
application_preferences	通过	0.265
ffmpeg_disk_discovery	通过	0.084
application_resources	通过	0.045
portrait_storage_queue_v120	通过	36.855
compatibility_iphone_v121	通过	6.448
compatibility_blackmagic_v121	通过	5.188
media_pipeline	失败	29.726
color_signal	通过	14.229
ffmpeg_discovery_capabilities	通过	5.351
publication_no_overwrite	通过	5.461
sustained_4k120	通过	78.546

21.4 media_pipeline 为什么失败

失败断言位于 tests/integration.py 第 154 行。测试先由有效源文件复制生成一个只把 color_primaries 改成 bt709 的 MOV，保留 ProRes 与 HLG 条件，再期待转换拒绝且目标文件不存在。1.2.1 的 UnsupportedReasons 已不再检查 primaries，因此该文件被接受，生产转换按显式 BT.2020 NCL 解释处理并通过自身输出验证。测试于是报告 Unsupported input accepted。

这是一个可以由代码直接解释的契约冲突：旧集成测试仍期待更严格颜色标签准入，新版发布说明则主动放宽格式门槛。当前单元测试已经包含可选标签缺失不阻断的断言，但较早端到端测试的这项没有同步。最直接的维护工作是先决定对“明确冲突的基色标签”和“缺失标签”是否应采用不同政策，再让实现、测试和文档一致。本文只分析，不擅自改写仓库策略。

21.5 失败不能被简单归为无害

从测试维护角度看，这是过时预期导致的失败；从信号解释角度看，它也暴露了宽松准入的实际代价。一个明确标为 BT.709 的 HLG 文件仍被当作 BT.2020 处理，若标签真实反映内容，则颜色可能错误。生产输出验证使用同一输入解释建立参考，无法独立证明源标签被忽略是合理的。这属于模型共同假设，不是输出格式检查能够消除的问题。

因此论文不把二十三项通过解释为“工具毫无问题”，也不把一项失败解释为“所有转换不可靠”。更准确的结论是：数学、并行、工具资格和多个媒体路径获得了本环境证据；当前完整集成套件在输入契约演进处存在未解决不一致，且该策略需要更清晰的用户提示和测试区分。由于 `integration.py` 在断言处停止，其后续变体在这次测试中未执行，不能计入本次已覆盖范围。

21.6 持续测试的具体结果

本次 `sustained_4k120` 使用两秒、240 帧、 3840×2160 、120 fps 的生成素材。转换耗时为 43.449496 秒，对应约 5.523654 fps；应用峰值私有内存为 15142912 字节，约 14.44 MiB。三帧抽样位置为 0、119、239，共 1536 个亮度样本，最大编码后误差为 0.86505094 码，低于该测试的两码阈值。

这个转换耗时不等于 CTest 对该用例报告的 78.55 秒，因为后者还包含夹具生成、额外抽样与脚本开销。它也不意味着实时处理 4K120：输入采集速率是一百二十帧每秒，实际离线转换速度约五点五帧每秒。两个“fps”描述不同概念，必须在图表标题和坐标中分清。

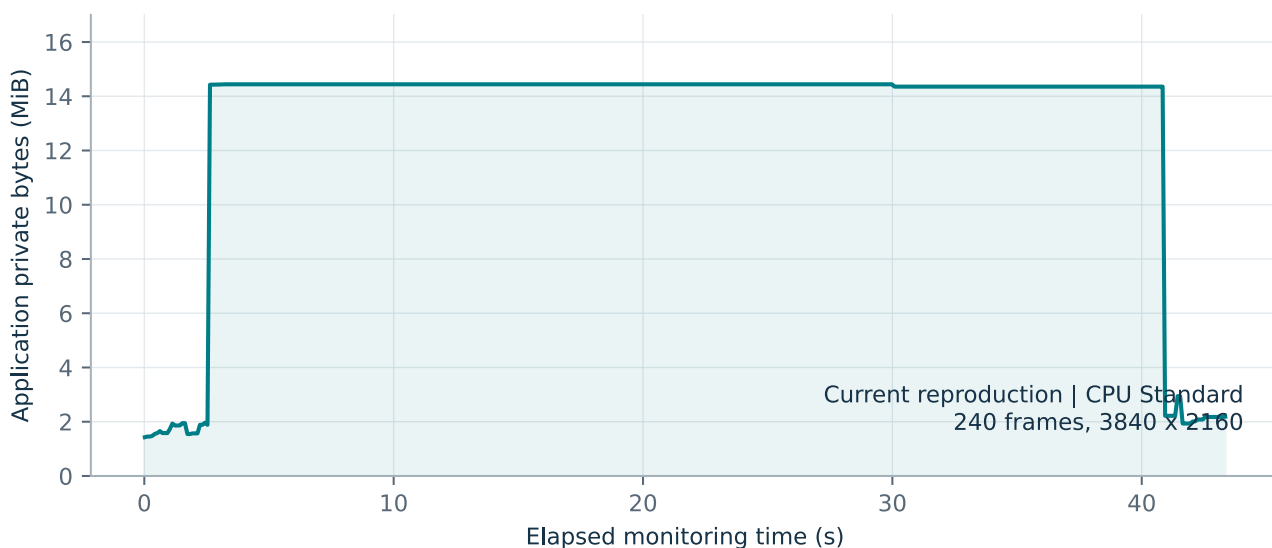


图 16 本文复测的 240 帧 4K120 CPU 标准模式应用私有内存曲线。数据来自 `sustained-report.json`；仅计主应用，不包含 FFmpeg 子进程或 GPU 显存。曲线是一次运行观察。

本章主要源码依据：[CMakeLists.txt](#)；[tests/integration.py](#)；[tests/sustained.py](#)；[docs/VALIDATION.md](#)。

第二十二章 性能数据、复杂度与可重复比较

22.1 历史矩阵的工作负载

docs/benchmarks/1.2.0.json 记录二十个正式案例：1080p24 与 4K 的 24、30、60、120 fps，各自组合标准 / 创意和 CPU / CUDA。输入是一秒生成灰阶斜坡，每次任务都进行工具检查和生产输出验证，没有添加音频或旋转。因此它适合比较该机器上的颜色后端与工作负载成本，不适合推断长片、复杂彩色纹理、磁盘阵列或热稳定后的吞吐。

硬件记录为二十逻辑 CPU、RTX 3080 Ti Laptop GPU、十六 GB 显存与指定 NVIDIA 驱动。当前论文将其绘制为历史测量图，并明确日期与版本。一个文件名叫 1.2.0 的数据集不能在图标题中被改成“1.2.1 新实测”，即使颜色核未改变，线程、操作系统状态和外围处理也可能影响时间。

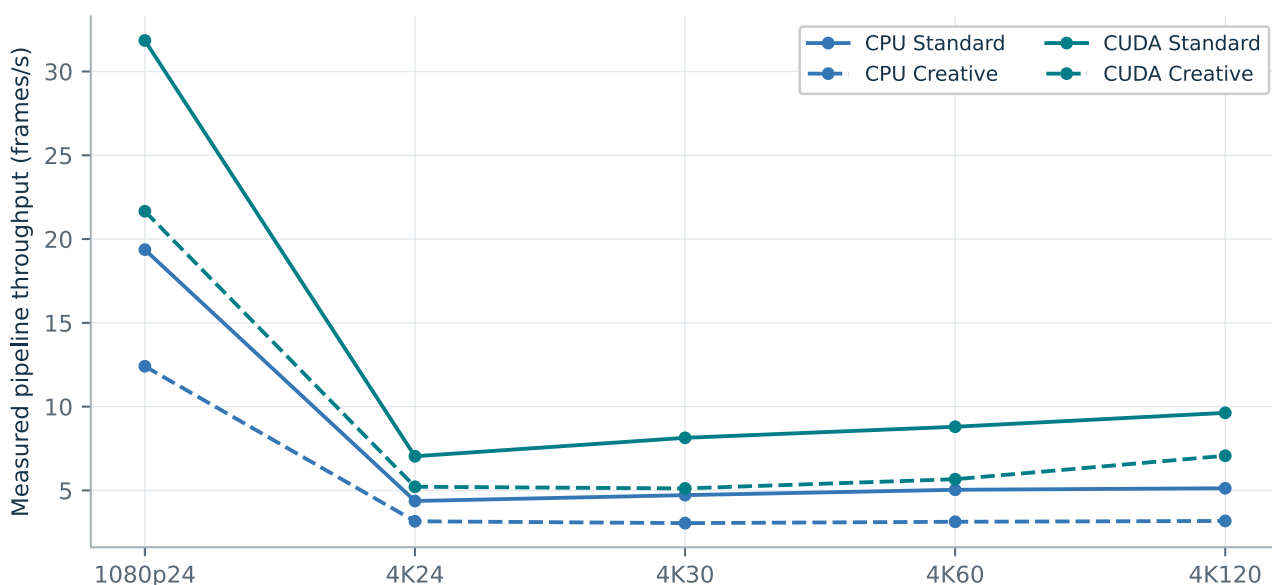


图 17 仓库 1.2.0 历史吞吐矩阵。实线 / 虚线区分标准与创意，颜色区分 CPU 与 CUDA；纵轴为实际流水线处理帧率，不是输入拍摄帧率。数据为单机单次案例。

22.2 从历史数字计算加速比

4K120 标准模式的历史流水线耗时为 CPU 23.387 秒、CUDA 12.465 秒，比例约 1.876。对应 CLI 总耗时为 27.336 与 16.405 秒，端到端加速约 1.666，低于流水线加速。原因包括启动、资格、元数据和验证等未同比缩短的成本。展示加速比时必须写明分母和计时范围。

$$S_{\text{pipeline}} = \frac{23.387}{12.465} \approx 1.876, \quad S_{\text{CLI}} = \frac{27.336}{16.405} \approx 1.666 \quad (64)$$

同一输入的创意历史流水线为 37.734 对 16.985 秒，比例约 2.222。这个数高于标准模式并不说明创意必然更快，而是 CPU 创意路径本身更重，CUDA 加速了更多计算。绝对耗时与相对加速是两种指标，最好同时提供，避免只选高比例数字造成误解。

22.3 Amdahl 定律与瓶颈迁移

若总工作中可加速部分比例为 p ，局部加速倍数为 s ，忽略额外成本时整体加速上限由下式给出。对 LogForge 而言，ProRes 编解码、矩阵滤镜、容器和验证仍在 CPU 或磁盘侧；即使颜色核无限快，剩余部分仍限制总体速度。实际 GPU 上传下载又使上限更低。

$$S_{\text{total}} \leq \frac{1}{(1-p) + p/s} \quad (65)$$

流水线重叠也会改变如何理解 p ：阶段并非全部串行相加，某阶段缩短可能只减少一部分墙钟时间。报告中的 read、write 包含背压，decoder / encoder CPU seconds 是累积 CPU 用时，GPU event time 是设备时间，不能把这些数直接相加当作总耗时。本文保留每一种时间的定义，而不制作错误的相加饼图。

22.4 内存比较必须包含范围

历史 4K120 标准 CPU 应用峰值约 14.4 MiB，但进程树约 2110.7 MiB；CUDA 标准应用约 297.9 MiB，进程树约 2768.5 MiB。创意 CUDA 应用约 565.6 MiB，进程树约 2866.2 MiB。这些数字提醒读者：小型原生 GUI 的进程占用很低，不代表整套转码只需十几 MiB。

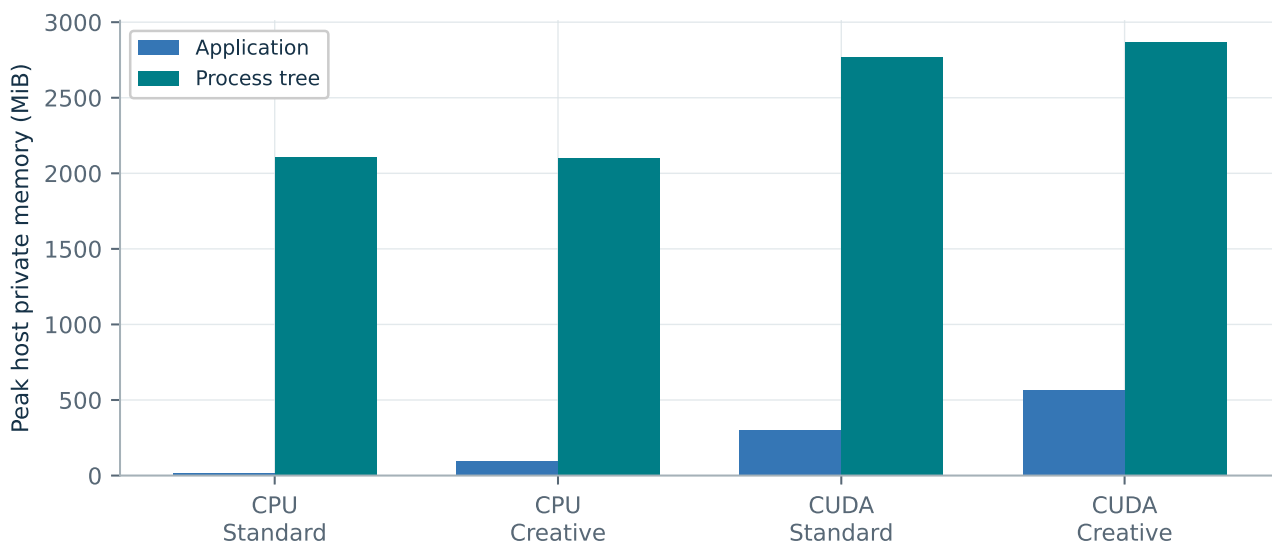


图 18 1.2.0 历史 4K120 四种配置的应用与进程树峰值内存。两组柱使用相同单位，GPU 设备缓冲另有报告，不能把主机私有内存当作显存统计。

理论缓存上界与实测私有字节应相互核对而非相互替代。标准桥的十二 MiB 能解释主应用的主要像素缓存，但线程栈、DLL、驱动和分配器会增加开销。复杂度表达式说明随输入规模的趋势，实测值说明特定环境中的常数，二者共同才构成可靠性能解释。

22.5 复现实验的改进方案

若要得出跨设备结论，应重复运行并报告中位数、分位数和离散度，记录电源模式、温度、后台负载、磁盘与驱动。输入应包含平滑斜坡、噪声、强纹理、彩色锐边与真实相机片段，分别报告生成、验证和转换时间。长时间测试应区分冷启动、稳定状态和温度降频，而非把一秒片段的结果线性外推到一小时。

本文没有生成不存在的置信区间，也没有把单次新测试与单次历史测试的小差异解释成版本性能提升。当前复测约 43.45 秒，历史重叠桥约 45.20 秒，环境噪声与测量条件足以影响这种差别；可以报告两者，但不足以证明软件优化。这个克制使性能章节的结论可被独立复现者评估。

本章主要源码依据：[docs/BENCHMARK_1.2.0.md](#)；[docs/benchmarks/1.2.0.json](#)；[tests/benchmark.py](#)。

第二十三章 从一帧样本到最终文件的实现演练

23.1 建立一个明确的输入实例

考虑一个 3840×2160、二十四帧每秒、两秒时长的 ProRes HQ HLG MOV，使用 BT.2020 NCL、视频范围、left 色度位置，带九十度显示方向和 48 kHz 双声道 PCM。设视频从零开始，音频从 500 / 48000 秒开始；曝光为零，创意关闭，后端强制 CPU。该实例是教学构造，不声称对应某个未公开相机原片。

首先 Probe 解析出媒体摘要，格式准入通过。方向矩阵若属于受支持标准旋转，则输出几何变为 2160×3840。源包扫描应得到四十八帧及固定周期；输出空间预检、日志可写检查通过后才创建 partial。这个阶段尚未改变像素，任何拒绝都不会产生最终输出文件。

23.2 追踪一个中性样本

选择接近百分之十八灰的 HLG 中性像素。若 H 精确采用参考值 0.3782588831，逆函数得到约 0.047693 场景相对量，乘 S 后回到 0.18，再编码得到 P 约 0.4882724585。三个通道相等，因此输出色差为零，亮度未量化码约为 491.7267。实际输入整数只能逼近这个 H，所以具体解码输出还会有量化和压缩差异。

$$H \xrightarrow{E} E(H) \xrightarrow{S} 0.18 \xrightarrow{f} 0.4882724585 \xrightarrow{64+876P} 491.7267 \quad (66)$$

这条链应作为数值理解，而不是要求每个真实像素最终等于 492。色度重建会受邻域影响，ProRes 会引入压缩误差，误差扩散还可能使相邻像素落在不同整数。只有均匀夹具、确定取样位置和明确误差指标，才能把这个教学链变成有效测试。

23.3 追踪一块内存

解码器依次输出旋转后的 G、B、R 平面。标准模式第一块最多包含一百万余个浮点分量，读线程将其置为准备态；颜色线程组分片完成变换并更新统计；写线程把它交给编码器。第二、第三块可在不同阶段并行，块之间仍严格保持字节顺序。总样本数最终应为 2160×3840×3×48。

如果最后一块只有不完整的 float，读阶段拒绝；如果样本数不是完整帧倍数，桥结束时拒绝；如果帧数不是四十八，任务退出检查拒绝。这些检查分别观察字节、帧结构和媒体时间契约。重复分层并非冗余，因为每一层能捕获不同形式的损坏。

23.4 追踪音频与容器

编码器从管道取得视频，从第二源输入复制音频。视频新时钟按已验证的二十四帧每秒建立，音频保留相对视频的 1 / 96 秒起点。源 movie timescale 交给输出封装器，创建头字段稍后恢复，时间码若存在则重建。竖拍方向已经作用于像素，所以最终视频显示矩阵应为单位矩阵。

编码后写入 logs，检查 nclc 9 / 2 / 9，重新 Probe 并扫描输出包。PixelSanity 在选中的三帧小块比较标量参考与实际解码结果，信号统计报告是否有下限截断或超名义白。只有这些条件和报告保存均成立，partial 才被移动为用户指定名称。

23.5 追踪一个 GPU 失败分支

将同一实例改为 Auto，如果初始化失败则一开始选择 CPU；如果第若干块的 CUDA 运算失败，在提交之前原主机块仍保持 HLG 输入，Auto 将该块交给 CPU，然后后续块继续 CPU。报告记录 `fallback_reason` 和 `cpu after cuda`，不能继续把全片标为纯 CUDA 成功。若用户选强制 CUDA，同一异常会结束任务而不会静默降级。

这个分支说明后端选择属于结果可追溯信息。实际工作可能跨过两种实现，只要块级提交与参考资格成立，颜色契约仍可保持；但性能报告必须反映混合执行。把异常吞掉并只显示“完成”会失去诊断信息，也让用户无法解释同样设置为何耗时不同。

23.6 追踪一个最终路径冲突

若另一进程在转换中创建了目标文件，早期不存在检查已经失效。最终 `MoveFileExW` 不允许替换，任务报 `OutputExists`，保留外部新文件并清理本任务临时媒体。这时可以有一份验证通过的报告，但没有本任务发布的最终视频；所以报告里的 `passed` 表示媒体验证，不应被外部脚本误当作最终文件发布保证。

这也是设计接口时值得进一步细化的地方：目前报告在发布前保存，发布失败信息主要留在任务错误和日志中。若未来增加明确 `publication` 状态或补写事务结果，自动化消费者更容易区分“内容通过但发布失败”和“全部完成”。本文将其作为改进建议，而非声称当前 JSON 已实现此字段。

本章主要源码依据：`src/Transcode.cpp`；`src/FloatBridge.cpp`；`src/color/CudaTransformer.cpp`。

第二十四章 讨论、实现局限与研究结论

24.1 能够成立的核心论证

本文的第一项论证是数学一致性。HLG 逆函数、参考白尺度、曝光和 Apple Log 编码按明确顺序组合，创意模式另有独立开关和亮度模型。分段导数、灰卡固定点、饱和度亮度保持以及创意中性曲线的正导数下界都能从公式直接推导；舍入接点、负值扩展和下限平台则给出必要限定。数学结论与源码函数逐一对应，使读者可以把符号与实现互相检查。

第二项论证是执行结构一致性。标准变换逐分量独立，允许有界分块；创意变换依赖三通道，要求正确配对；CPU tile 与 CUDA 网格均保持这种依赖结构。CUDA 只有在完全同步和检查后提交，使 Auto 回退不重复变换；三槽状态机通过所有权转移保持顺序。这些结构性质比仅展示一次正确输出更有解释力。

第三项论证是交付约束。时间戳验证为无时间戳管道提供前提，音频相对时间与媒体元数据有明确检查，MOV 写入限制结构以维持媒体偏移，工具租约绑定被执行字节，最终文件发布依赖验证和无覆盖移动。每一项都处理一个具体失效模式，共同构成媒体任务的工程正确性，而不是单一颜色公式的附属装饰。

24.2 当前证据不能推出的命题

本文不能证明输出恢复原始传感器动态范围、完全等价于原生相机 Apple Log、保留所有细纹理、兼容所有编辑器或实时处理高帧率 4K。这些命题要么与不可逆信息损失冲突，要么需要未执行的成对拍摄、视觉评测或跨设备实验。仓库也没有提供足以支持它们的完整证据。

同样，当前源码没有形式化验证全部 C++ 并发和所有 Win32 错误分支。本文给出的是基于可检查实现结构的条件化证明、数值推导和测试证据，不是机械证明助手生成的全程序定理。对图表中由公式计算的曲线、历史单次测量和本文单次复测，均应按各自证据类型理解。

24.3 最需要澄清的输入契约

1.2.1 放宽准入有明确兼容性目的，但需要把缺失信息、明确冲突信息和不可解释信息分开。缺少相机型号不应影响颜色计算；缺少色度位置时默认 left 可能是务实选择；明确声明其他基色却强制 BT.2020 则具有不同风险。一个统一“可选标签”分类不足以表达这些差异。

可考虑在保持兼容性的同时记录默认项清单、对明确冲突提示用户、为不同策略设置稳定报告字段。相应测试应覆盖“缺失允许”“冲突警告或拒绝”“用户明确覆盖”的独立案例。本文失败的集成测试提供了具体起点：先修订契约，再同步代码和测试，而不是为了全绿直接删掉断言或恢复旧门槛。

24.4 更强的科学验证方案

若研究目标转向相机外观等价，需要同场景、同曝光、同白平衡、同镜头与可追溯设置的 HLG / 原生 Log 成对捕获，使用灰阶、色卡和高动态场景，统一解码范围与显示变换后比较。应报告亮度、色差、噪声和局部纹理指标，并说明相机模式是否共享 ISP 路径。只比较两个播放器窗口的截图不能隔离这些变量。

若目标是编辑器互操作，应按软件、版本、色彩管理模式和数据级别建立矩阵，包含负基线、仅 logs、其他候选字段和真实输出；自动识别与渲染数值都需检查。若目标是性能，应使用重复长任务、不同纹理和设备；若目标是可靠性，应扩大故障注入、文件系统异常和恶意容器结构覆盖。不同研究问题需要不同实验，不能由一个“转换成功”统一回答。

24.5 结论

LogForge 展示了一种可理解的工程路径：以公开颜色数学定义信号转换，以独立多媒体工具完成编解码，以有限内存和明确所有权组织并发，以结构化验证决定交付。它的价值不在于创造一种新的 Log 曲线，而在于将已有标准与实际桌面工作流的时间、元数据和失败行为连接起来，并留下可检查的证据。

在本次固定版本研究中，Release 构建与二十三项测试通过，一项旧输入拒绝预期与新版准入冲突，持续 4K120 数值抽样符合现有阈值。这支持核心转换和多个可靠性机制在本环境中按设计工作，也要求保留输入解释、文档演进和有限抽样的限制。严格的实现论文既要解释成功为何成立，也要说明结论从哪里开始不再成立；本文据此完成从公式、代码、测试到最终文件的逐层论证。

本章主要源码依据：[README.md](#)；[docs/RELEASE_1.2.1.md](#)；[docs/VALIDATION.md](#)。

附录 A 符号、单位与复核指南

A.1 统一符号表

表 4 主要符号与单位

符号	含义	单位或定义域
H	HLG 编码分量	归一化；名义零到一
E	HLG 逆 OETF 场景相对量	相对量
S	参考白归一化尺度	约 3.7741181185
e	曝光调整	EV, CLI 负八到正八
R、G、B	曝光后场景线性分量	参考反射率坐标
P	Apple Log 编码分量	归一化
Y、Cb、Cr	非线性矩阵分量	归一化
q	存储前或解码后的码值	十位码单位
W、H（几何上下文）	画幅宽、高	像素；与信号 H 按上下文区分
tau、T	时间基、候选包周期	秒每 tick、tick
N	帧数或样本数	按下标明确

A.2 公式复核的顺序

复核者可以从 Color.h 常数开始，先检验 Apple Log 的零、接点、中灰和高端，再检验 HLG 在二分之一处的值与导数，随后检验参考白和百分之十八灰的组合映射。完成这三个步骤后，再加入 YCbCr 的偏置与尺度。这个顺序把传递函数错误、曝光错误和范围错误分离，避免面对整条管线时无法定位差异。

创意模式应先关闭饱和度变化，只使用中性灰阶检验单调性与固定点，再加入彩色块检查加权亮度关系。边界测试要包含负值、零、接点两侧、超白、极端曝光和非有限输入；测试正常范围的均匀随机数不能保证覆盖这些稀有但重要的位置。计算图表时也应使用与论文声明一致的常数精度。

A.3 源码复核的顺序

从 CLI 或 MainWindow 进入 TranscodeJob，再沿 ToolTrust、Probe、Cadence、FloatBridge、FloatTransformer / CudaTransformer、MetadataWriter、PixelSanity 和发布动作阅读，能够把每个模块放入实际生命周期。只按字母顺序浏览文件容易漏掉“谁持有租约”“谁拥有 partial”“何时可以覆盖样本”等跨文件条件。

附带全仓库清单给出每个受控文件的路径、类别、行数和 SHA-256，并对生成 PTX 与第三方 JSON 单头文件单列。清单哈希基于本地文件实际字节；Git 换行转换可能使其不同于仓库对象内容哈希，所以它不是 Git blob ID。读者应先确认提交号，再根据清单核对本次分析材料，避免把两个不同版本的源码和测试混合。

A.4 实验记录的读取方式

本次回归 XML 是当前运行结果；持续测试 JSON 包含转换耗时、应用私有内存序列和抽样误差；历史性能 JSON 来自仓库 1.2.0。将两组数据分开放置，是为保留时间和版本来源。图中的理论曲线由排版脚本直接根据公式生成，不是从相机拍摄数据拟合，也不伪装成实际波形仪读数。

论文配套 LaTeX 与 Markdown 用于编辑和复用，PDF 由本地矢量排版程序生成。当前内置 LaTeX 编译器因平台目录错误无法验证完整工程，故不把源稿标为已经由该编译器成功构建。公式在 PDF 中以矢量路径呈现，正文可搜索，图表附有编号、单位、数据来源和适用范围。

附录 B 全仓库文件与研究章节对应

以下索引覆盖固定提交中的全部受版本控制文件。它说明文件在实现、测试、构建或文献证据中的角色，并以逐文件行数与哈希清单作为可复核补充。第三方库、生成代码、图片和许可不按项目手写算法处理。索引不是“每行程序均已形式化证明”的声明，而是论文材料范围与实现责任的追踪表。

表 5 受控文件范围，共 102 项

文件路径与行数	实现责任 / 研究用途
.clang-format 9 行	仓库格式、换行和忽略规则；复现基础
.gitattributes 10 行	仓库格式、换行和忽略规则；复现基础
.github/workflows/build-windows.yml 71 行	C++20/MSVC目标、测试注册、构建与发布允许列表；第8、21章
.gitignore 48 行	仓库格式、换行和忽略规则；复现基础
CHANGELOG.md 141 行	项目说明、历史证据或开发流程；按章节主题交叉核对
CMakeLists.txt 124 行	C++20/MSVC目标、测试注册、构建与发布允许列表；第8、21章
CONTRIBUTING.md 19 行	项目说明、历史证据或开发流程；按章节主题交叉核对
LICENSE 21 行	项目或第三方许可与归属说明；发行边界
README.md 233 行	项目说明、历史证据或开发流程；按章节主题交叉核对
THIRD_PARTY_NOTICES.md 41 行	项目或第三方许可与归属说明；发行边界
docs/APPLE_LOG_IDENTIFICATION.md 153 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/ARCHITECTURE.md 147 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/BENCHMARK_1.2.0.md 62 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/BUG_AUDIT_1.1.1.md 63 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/COLOR_PIPELINE.md 97 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/CREATIVE_ADJUSTMENTS.md 46 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/FFMPEG_PROVIDER.md 54 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/ICON.md 28 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/METADATA.md 79 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/RELEASE_1.2.0.md 92 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/RELEASE_1.2.1.md 46 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/TECHNICAL_REFERENCES.md 137 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/VALIDATION.md 598 行	项目说明、历史证据或开发流程；按章节主题交叉核对

文件路径与行数	实现责任 / 研究用途
docs/benchmarks/1.2.0.json 2805 行	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/images/LogForge.png 二进制 37255 字节	项目说明、历史证据或开发流程；按章节主题交叉核对
docs/verification/1.2.0.json 320 行	项目说明、历史证据或开发流程；按章节主题交叉核对
include/logforge/AppleLogIdentification.h 14 行	35字节识别扩展写入及结构检查；第15章
include/logforge/Color.h 53 行	颜色模型接口、常数、参数与统计结构；第3至7章
include/logforge/CudaTransformer.h 24 行	动态驱动接口、资格验证和事务提交；第11章
include/logforge/Discovery.h 61 行	本地卷与目录遍历；第17章
include/logforge/FFmpeg.h 62 行	工具能力、发现调度和固定下载；第17章
include/logforge/FloatBridge.h 21 行	三槽读算写状态机与线程预算；第9章
include/logforge/FloatTransformer.h 18 行	持久 CPU 工作线程、tile 配对及统计合并；第10章
include/logforge/Localization.h 34 行	稳定消息键和单遍参数替换；第20章
include/logforge/Media.h 87 行	探测解析、准入、元数据参数与输出验证；第2、16章
include/logforge/Messages.inc 262 行	双语消息宏表、稳定键与占位符；第20章
include/logforge/PixelSanity.h 24 行	首中尾帧九块编码后粗差检查；第7章
include/logforge/Platform.h 115 行	宽字符、句柄、进程监督、管道与哈希；第18章
include/logforge/Queue.h 8 行	串行多文件任务及逐项失败记录；第20章
include/logforge/Settings.h 23 行	字段合并、并发锁及原子偏好保存；第19、20章
include/logforge/StorageSafety.h 18 行	文件身份日志、状态锁与所有权恢复；第19章
include/logforge/ToolTrust.h 29 行	双可执行文件身份、批准与租约；第17章
include/logforge/Transcode.h 33 行	完整媒体任务、后端、验证报告与最终发布；第8、23章
include/logforge/Ui.h 49 行	主题、DPI、原生绘制与截图；第20章
resources/LogForge.ico 二进制 103909 字节	图标、清单、版本资源及生成模板；构建与界面
resources/LogForge.manifest 9 行	图标、清单、版本资源及生成模板；构建与界面
resources/LogForge.png 二进制 1161472 字节	图标、清单、版本资源及生成模板；构建与界面
resources/LogForge.rc 34 行	图标、清单、版本资源及生成模板；构建与界面
resources/Version.h.in 6 行	图标、清单、版本资源及生成模板；构建与界面
resources/VersionResource.h.in 4 行	图标、清单、版本资源及生成模板；构建与界面
src/AppleLogIdentification.cpp 194 行	35字节识别扩展写入及结构检查；第15章
src/Cadence.cpp 168 行	全包周期、相位与固定时钟验证；第12章

文件路径与行数	实现责任 / 研究用途
src/Cli.cpp 223 行	命令与数值参数、核心自动化入口；第20章
src/Discovery.cpp 141 行	本地卷与目录遍历；第17章
src/DiscoveryHelper.cpp 501 行	受监督隐藏搜索入口与来源预算；第17章
src/FFmpeg.cpp 416 行	工具能力、发现调度和固定下载；第17章
src/FFmpegNumeric.cpp 234 行	独立整数信号与实际编解码数值资格；第5、7章
src/FloatBridge.cpp 183 行	三槽读算写状态机与线程预算；第9章
src/Localization.cpp 77 行	稳定消息键和单遍参数替换；第20章
src/MainWindow.cpp 1710 行	单后台工作者、事件与可测量布局；第20章
src/Media.cpp 400 行	探测解析、准入、元数据参数与输出验证；第2、16章
src/MediaSafety.cpp 272 行	时间语义、显示矩阵和固定宽度头修补；第13、16章
src/MetadataPolicy.cpp 53 行	元数据白名单和冲突分类；第16章
src/MovAnalyzer.cpp 433 行	有界原子解析与类型化语义比较；第14章
src/PixelSanity.cpp 184 行	首中尾帧九块编码后粗差检查；第7章
src/Platform.cpp 398 行	宽字符、句柄、进程监督、管道与哈希；第18章
src/Queue.cpp 49 行	串行多文件任务及逐项失败记录；第20章
src/Settings.cpp 120 行	字段合并、并发锁及原子偏好保存；第19、20章
src/SettingsWindow.cpp 659 行	设置草稿、保存取消、候选审阅与对话框；第20章
src/StorageSafety.cpp 214 行	文件身份日志、状态锁与所有权恢复；第19章
src/ToolTrust.cpp 124 行	双可执行文件身份、批准与租约；第17章
src/Transcode.cpp 584 行	完整媒体任务、后端、验证报告与最终发布；第8、23章
src/Ui.cpp 322 行	主题、DPI、原生绘制与截图；第20章
src/color/AppleLog.cpp 19 行	Apple Log 分段编解码与公开常数契约；第4章
src/color/ColorKernel.cu 58 行	设备端double颜色核与共享内存归约；第11章
src/color/ColorKernel.manifest.json 14 行	CUDA源、PTX、编译器身份与精确参数记录；第11章
src/color/ColorKernel.ptx.h 1620 行	生成PTX字节数组：检查内容哈希及源清单，不视为手写算法；第11章
src/color/CudaTransformer.cpp 333 行	动态驱动接口、资格验证和事务提交；第11章
src/color/FloatTransformer.cpp 127 行	持久 CPU 工作线程、tile 配对及统计合并；第10章
src/color/HLG.cpp 98 行	HLG 逆变换、参考尺度、创意调整和信号计数；第3、6章
tests/application.cpp 262 行	偏好、发现和版本资源测试；第21章

文件路径与行数	实现责任 / 研究用途
tests/benchmark.py 97 行	正式吞吐矩阵与过程观测；第22章
tests/compatibility_v121.py 145 行	竖拍像素、矩阵与音频偏移回归；第13、21章
tests/cuda_artifact.py 18 行	CUDA源码、嵌入PTX哈希与选项一致性；第11章
tests/fake_ffmpeg.cpp 10 行	未知工具不得执行的标记夹具；第17章
tests/gui_smoke.py 123 行	交互桌面GUI场景脚本，本文未重跑；第20章
tests/hardening.cpp 367 行	节奏、工具信任、MOV、并行与发布回归；第21章
tests/identification.cpp 101 行	识别写入、幂等与失败结构测试；第15、21章
tests/integration.py 255 行	端到端合成媒体测试；本次在第154行旧准入断言失败；第21章
tests/layout_snapshots.py 35 行	生产布局与主题DPI截图断言；第20章
tests/package_audit.py 38 行	归档允许列表、CRC、EXE与SHA-256审计；附录
tests/process_fixture.cpp 51 行	后代持管道、长行、退出码夹具；第18章
tests/reliability.cpp 246 行	进程、发现与CLI异常回归；第21章
tests/resolve_identification.py 110 行	真实Resolve控制导入脚本，本文未重跑；第15章
tests/signal_integrity.py 160 行	独立整数色块、创意与范围计数；第7、21章
tests/sustained.py 90 行	240帧4K120、内存和编码后亮度；第21章
tests/tests.cpp 396 行	基础数学、媒体和平台测试；第21章
tests/v120.cpp 386 行	存储、矩阵、CUDA故障、队列和媒体回归；第21章
third_party/NVIDIA-CUDA-NOTICE.txt 23 行	项目或第三方许可与归属说明；发行边界
third_party/nlohmann/LICENSE.MIT 21 行	项目或第三方许可与归属说明；发行边界
third_party/nlohmann/json.hpp 25526 行	第三方JSON 3.12.0单头依赖：JSON读写支持，非项目颜色算法
tools/compile_cuda.py 48 行	NVRTC精确选项编译和生成清单；第11章

参考文献与源码定位

正文的实现论述以固定提交源码为主要依据，章节末的源码定位列出主要文件；下面给出完整的项目资料和关键外部原始参考。外部文档用于核对公开定义与接口，仓库历史实验用于说明当时观察；二者均不替代本文实际执行结果。网页核对日期为 2026 年 9 月 29 日。

[1] 项目固定版本源码与README。

<https://github.com/HongHuo0728/LogForge/tree/6b43a93fbd4d1457ff20b4c99a030c0e398f520a>

[2] LogForge COLOR_PIPELINE：数值契约。

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/COLOR_PIPELINE.md

[3] LogForge CREATIVE_ADJUSTMENTS：项目自定义调整。

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/CREATIVE_ADJUSTMENTS.md

[4] LogForge ARCHITECTURE：架构说明，含历史政策。

<https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/ARCHITECTURE.md>

[5] LogForge RELEASE_1.2.1：当前准入与像素方向策略。

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/RELEASE_1.2.1.md

[6] LogForge VALIDATION：版本化历史验证记录。

<https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/VALIDATION.md>

[7] LogForge BENCHMARK_1.2.0：历史性能矩阵。

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/BENCHMARK_1.2.0.md

[8] LogForge APPLE_LOG_IDENTIFICATION：原生参考与Resolve受控实验。

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/APPLE_LOG_IDENTIFICATION.md

[9] LogForge METADATA 与 FFMPEG_PROVIDER：容器与工具政策。

<https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/METADATA.md>

[10] LogForge TECHNICAL_REFERENCES：外部原始参考索引。

https://github.com/HongHuo0728/LogForge/blob/6b43a93fbd4d1457ff20b4c99a030c0e398f520a/docs/TECHNICAL_REFERENCES.md

参考文献与源码定位（续）

- [11] Apple提供的ACES IDT，固定提交528c78fe：Apple Log解码常数。
https://github.com/ampas/aces-dev/blob/528c78fe2c0f4e7eb322581e98aba05de79466cb/transforms/ctl/idt/vendorSupplied/apple/IDT.Apple.AppleLog_BT2020.ctl
- [12] ITU-R BT.2100-3 (2025)：HLG传递函数；本次核对官方版本索引。
<https://www.itu.int/rec/R-REC-BT.2100>
- [13] ITU-R BT.2408-6：HDR制作参考；本文尺度依据仓库固定数值契约交叉说明。
https://www.itu.int/dms_pub/itu-r/opb/rep/R-REP-BT.2408-6-2023-PDF-E.pdf
- [14] ITU-R BT.2020：超高清电视基色与矩阵。
<https://www.itu.int/rec/R-REC-BT.2020>
- [15] FFmpeg官方滤镜手册：zscale；具体运行行为由资格测试约束。
<https://ffmpeg.org/ffmpeg-filters.html#zscale>
- [16] FFmpeg n8.1.2 MOV封装源文件：仓库引用的固定版本。
<https://github.com/FFmpeg/FFmpeg/blob/n8.1.2/libavformat/movenc.c>
- [17] Apple QuickTime File Format：矩阵与容器原始文档。
<https://developer.apple.com/documentation/quicktime-file-format/matrices>
- [18] NVIDIA NVRTC 12.8.1：生成PTX所用工具接口参考。
<https://docs.nvidia.com/cuda/archive/12.8.1/nvrtc/index.html>
- [19] Microsoft Windows：Job Objects监督接口参考。
<https://learn.microsoft.com/en-us/windows/win32/procthread/job-objects>
- [20] CMake CPack：归档与校验和生成参考。
<https://cmake.org/cmake/help/latest/module/CPack.html>